



UNIVERSITE DE FIANARANTSOA

ECOLE NATIONALE D'INFORMATIQUE

MEMOIRE DE FIN D'ETUDES POUR L'OBTENTION DU DIPLOME DE MASTER PROFESSIONNEL

Mention : Intelligence Artificielle
Parcours : Objets Connectés et Cybersécurité

Intitulé :

Mysa : une architecture autonome de cybersécurité comportementale pour les écosystèmes de maisons intelligentes

Présenté le : 23 Novembre 2026

Par : Mademoiselle **SAMARTINA** Eldinha Reinitha

Membres du Jury :

Président : Monsieur, RAZJNFJNRDK JNFKD, Docteur

Examinateur : Monsieur, hfhdhfdhfi

Rapporteurs : Madame, **RATIANANTITRA** Volatiana Marielle, Maître de Conférences

Monsieur, **ANDRIANTSIALO** Elyah Frisco

Année universitaire 2025-2026

Curriculum Vitae

Nom : SAMARTINA

Prénoms : Eldinha Reinitha

Date et lieu de naissance : 18 Mai 2004 à Vohiposa

Adresse : 580A/3710 Ampitakely Fianarantsoa

Tel : 038 50 122 52

E-mail : reinithaeldinha@gmail.com

FORMATIONS ET DIPLOMES

2025-2026 : Deuxième année en master professionnel à l'ENI Fianarantsoa en parcours Objets Connectés et Cybersécurité, mention Intelligence Artificielle.

2024-2025 : Première année en master professionnel à l'ENI Fianarantsoa en parcours Objets Connectés et Cybersécurité, mention Intelligence Artificielle.

2023-2024 : Troisième année en licence professionnelle à l'ENI Fianarantsoa en parcours Informatique Générale, mention Informatique.

2022-2023 : Deuxième année en licence professionnelle à l'ENI Fianarantsoa en parcours Informatique Générale, mention Informatique.

2021-2022 : Première année en licence professionnelle à l'ENI Fianarantsoa en parcours Informatique Générale, mention Informatique.

2020-2021 : Diplôme de baccalauréat série C avec mention assez bien au Lycée Sacré Cœur Ambohimasoa.

STAGES ET EXPERIENCES PROFESSIONNELLES

01 Juin 2026 - Maintenant : Technicienne en Cybersécurité au sein du groupe Rajaballi Antananarivo dont la mission consiste en l'analyse de vulnérabilités, l'analyse de risque et la remédiation, l'investigation d'incidents et la mise en place de solutions de sécurité.

01 Décembre 2025 - 31 Mai 2026 : Stage en IoT au sein de l'UN-IT Fianarantsoa dont le projet consiste en la mise en place d'un système de pointage par empreinte digitale relié au CRM interne.

23 Septembre 2024 - 30 Novembre 2024 : Stage d'administrateur systèmes et réseaux au sein du Cabinet EXTER Antananarivo dont le projet consiste en la centralisation de données par la mise en place d'un serveur de fichiers sécurisé.

Septembre 2023 - Novembre 2023 : Stage de développeur au sein du CIDST Antananarivo, au cours duquel la conception et la réalisation de la gestion des outils informatiques ont été effectuées.

COMPETENCES EN INFORMATIQUE

Systèmes d'exploitation : Windows, Linux, Windows Server

Bureautique : Microsoft Word, Excel, PowerPoint

Langages de programmation : Python, Java, C, C++ et C#

Technologies Web : HTML, CSS, JavaScript et PHP

Frameworks : Django, ReactJS, NodeJS et FastAPI

Systèmes de gestion de base de données : MySQL, PostgreSQL, SQLite

Outils : VMware, GNS3, Docker, Wireshark, Nmap, BurpSuite, Nikto, …

Systèmes embarqués : ESP32, PlatformIO (framework Arduino), MQTT

Méthodes de conception : Merise et 2TUP

Langage et méthode de conception : UML et Agile

Cybersécurité : analyse de vulnérabilités, évaluation de risque et remédiation, investigation d'incidents, test d'intrusion, …

Intelligence Artificielle : apprentissage automatique (scikit-learn, Isolation Forest), détection d'anomalies

CONNAISSANCES LINGUISTIQUES

Tableau 1 : Connaissances linguistiques

Langues	Comprendre à l'audition	Lire	Écrire	Parler et communiquer
Français	TB	B	B	AB
Anglais	AB	B	B	AB

Grille d'évaluation : TB : Très bien ; B : Bien ; AB : Assez bien ; P : Passable ; NS : Niveau scolaire.

CENTRES D'INTERET

Sport : Yoga

Loisirs et divertissements : séries, voyage, pâtisserie

Sommaire

Curriculum Vitae.....
Sommaire.....
Remerciements.....
Liste des figures.....
Liste des tableaux.....
Liste des abréviations.....
Introduction générale.....
Partie I : PRESENTATIONS.....
Chapitre 1 : Présentation de l'ENI.....
1.1. Information d'ordre général.....
1.2. Missions et historique.....
1.3. Organigramme institutionnel.....
1.4. Domaine de spécialisation.....
1.5. Architecture des formations pédagogiques.....
1.6. Relation de l'ENI avec les organismes externes.....
1.7. Débouchés professionnels des diplômés.....
1.8. Ressources humaines.....
Chapitre 2 : Présentation de l'établissement d'accueil.....
2.1. Description et statuts.....
2.2. Activités.....
2.3. Contacts et adresses.....
2.4. Partenaires.....
2.5. Organigramme.....
Chapitre 3 : Description du projet.....
3.1. Formulation.....
3.2. Objectif et besoins de l'utilisateur.....
3.3. Moyens nécessaires à la réalisation du projet (humains, matériels, logiciels).....
3.4. Résultats attendus.....
3.5. Chronogramme de travail.....
PARTIE II : ANALYSE ET CONCEPTION.....
Chapitre 4 : Analyse préalable.....
4.1. Analyse de l'existant.....
4.2. Critique de l'existant (points forts et points faibles).....
4.3. Proposition de solutions.....
Chapitre 5 : Modèle et architecture de l'IoT.....

5.1. Cadre conceptuel et fondements du modèle IoT.....
5.2. Conception et rôle de la couche perception (capteurs et dispositifs).....
5.3. Technologies de connectivité et protocoles de transmission dans la couche réseau.....
5.4. Modélisation schématique d'une architecture IoT appliquée.....
5.5. Analyse comparative des modèles de référence normalisés.....
5.6. Limites techniques et défis d'ingénierie de l'architecture IoT.....
Chapitre 6 : Études et choix des outils.....
6.1. Outils utilisés.....
6.2. Données et stratégie d'amorçage à froid.....
PARTIE III : RÉALISATION.....
Chapitre 7 : Présentation de l'environnement.....
7.1. Prototype de maison intelligente.....
7.2. Plateforme d'attaque.....
7.3. Présentation de Mysa.....
Chapitre 8 : Installation et configuration.....
8.1. Broker MQTT sécurisé.....
8.2. Segmentation réseau en VLAN.....
8.3. Firmware des nœuds ESP32.....
8.4. Ingestion MQTT et stockage.....
8.5. Moteur de détection.....
8.6. Modèles de référence et amorçage à froid.....
8.7. API de consultation et de commande.....
8.8. Dashboard de supervision.....
8.9. Caméra du téléphone.....
8.10. Notifications.....
8.11. Déploiement Docker et mode de repli.....
8.12. Lab d'attaque.....
Chapitre 9 : Test et validation.....
9.1. Protocole et mesures.....
9.2. Tests d'intrusion.....
9.3. Évaluation de la détection comportementale.....
9.4. Évaluation hors ligne des modèles de référence.....
9.5. Discussion et limites.....
Conclusion.....
Références bibliographiques.....
Références webographiques.....
Glossaire.....

Annexes.....
Table des matières.....
Résumé.....
Abstract.....

Remerciements

Au terme de ce travail, je tiens à exprimer ma sincère reconnaissance envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce mémoire.

- Monsieur HAJALALAINA Aimé Richard, Professeur, Président de l'Université de Fianarantsoa, pour avoir permis la tenue de cette formation de haut niveau au sein de l'établissement.
- Monsieur MAHATODY Thomas, Professeur, Directeur de l'Ecole Nationale d'Informatique, pour la qualité de l'encadrement administratif et pédagogique offert tout au long du cursus.
- Monsieur RAFANDROANA Patrice Mahenina, Directeur de la société UN-IT, pour m'avoir accueillie en stage et pour la confiance accordée dans la conduite de ce projet.
- Monsieur DIMBISOA William Germain, Maître de Conférences, Responsable de la mention Intelligence Artificielle, pour le suivi assuré durant toute l'année universitaire.
- Monsieur GILANTE Gesazafy, Assistant d'Enseignement Supérieur et de Recherche, Responsable du parcours Objets Connectés et Cybersécurité, pour les orientations apportées sur le contenu technique du mémoire.
- Madame/Monsieur [Nom et Prénoms], Président du jury, pour l'honneur fait d'évaluer ce travail.
- Madame/Monsieur [Nom et Prénoms], Examinateur, pour le temps consacré à la lecture et à la critique constructive de ce mémoire.
- Madame RATIANANTITRA Volatiana Marielle, Maître de Conférences, Encadreur pédagogique, pour la disponibilité, les conseils méthodologiques et le suivi régulier apporté pendant toute la durée du projet.
- Monsieur ANDRIANTSIALO Elyah Frisco, Co-directeur de la société UN-IT, Encadreur professionnel, pour l'accompagnement technique et les retours apportés sur la réalisation du prototype.
- Ma famille et mes amis, pour leur soutien moral constant tout au long de ce parcours.

Liste des figures

Figure 1 : Organigramme de l'Ecole Nationale d'Informatique.....
Figure 2 : Architecture des études correspondant au système LMD.....
Figure 3 : Organigramme de l'UN-IT.....
Figure 4 : Chronogramme de travail.....
Figure 5 : Architecture en couches de Mysa.....
Figure 6 : Segmentation du prototype en trois VLAN.....
Figure 7 : Plan de la maquette de démonstration (vue de dessus).....
Figure 8 : Architecture d'ensemble de Mysa (nœuds, hub, VLAN, notifications).....
Figure 9 : Flux de données de Mysa.....
Figure 10 : Capture des tests de refus et de succès de connexion.....
Figure 11 : Topologie GNS3 avec les trois VLAN.....
Figure 12 : Schéma de câblage du nœud esp32-sec-01 (la LED verte, sur GPIO 21, n'est pas représentée).....
Figure 13 : Schéma de câblage du nœud esp32-cmf-01.....
Figure 14 : Capture de la documentation interactive de l'API (/docs).....
Figure 15 : Dashboard — page « Maison » (état de la maison, pièces, score de risque).....
Figure 16 : Dashboard — page « Alertes » (épisodes filtrables, gravité, acquittement).....
Figure 17 : Dashboard — détail d'un appareil (courbes, causes du score).....
Figure 18 : Dashboard — page « Moteur » (état d'apprentissage de chaque flux).....
Figure 19 : Page caméra ouverte sur le téléphone.....
Figure 20 : Hub Mysa déployé dans Docker sous Windows.....
Figure 21 : Rapport du lab d'attaque exécuté depuis la machine Kali.....
Figure 22 : Alerte affichée sur le dashboard lors d'une attaque.....

Liste des tableaux

Tableau 1 : Connaissances linguistiques.....
Tableau 2 : Organisation du système de formation pédagogique de l'Ecole.....
Tableau 3 : Mentions et parcours au sein de l'ENI.....
Tableau 4 : Liste des formations existantes à l'ENI.....
Tableau 5 : Débouchés éventuels des jeunes diplômés.....
Tableau 6 : Grille de criticité (impact en lignes, probabilité en colonnes).....
Tableau 7 : Vulnérabilités identifiées.....
Tableau 8 : Évaluation des risques.....
Tableau 9 : Cartographie des composants et criticité.....
Tableau 10 : Points forts et points faibles de l'existant.....
Tableau 11 : Comparaison des deux solutions.....
Tableau 12 : Positionnement de Mysa par rapport aux modèles de référence.....
Tableau 13 : Alternatives pour le firmware ESP32.....
Tableau 14 : Comparaison de MQTT, CoAP et HTTP/REST.....
Tableau 15 : Alternatives pour le stockage.....
Tableau 16 : Jeux de télémétrie TON_IoT et catégories Mysa associées.....
Tableau 17 : Les trois régimes de fonctionnement d'un flux.....
Tableau 18 : Association entre nom de capteur et catégorie.....
Tableau 19 : Caractéristiques d'une fenêtre de 20 messages.....
Tableau 20 : Synthèse des outils retenus.....
Tableau 21 : Appareils du prototype.....
Tableau 22 : Blocs logiciels de Mysa et fichiers correspondants.....
Tableau 23 : Points d'accès de l'API.....
Tableau 24 : État d'avancement de la réalisation.....
Tableau 25 : Tests de vérification du broker.....
Tableau 26 : Plan de segmentation.....
Tableau 27 : Matrice de flux du réseau segmenté.....
Tableau 28 : Brochage du nœud esp32-sec-01.....
Tableau 29 : Tables SQLite.....
Tableau 30 : Règles du moteur.....
Tableau 31 : Corrélations entre capteurs.....
Tableau 32 : Détecteurs du modèle local.....
Tableau 33 : Paramètres du moteur.....
Tableau 34 : Pages du dashboard.....
Tableau 35 : Canaux de notification.....

Tableau 36 : Tests d’intrusion.....
Tableau 37 : Scénarios comportementaux.....
Tableau 38 : Résultats des tests d’intrusion.....
Tableau 39 : Détection et fausses alertes par scénario (6 scénarios × 5 graines).....
Tableau 40 : Fausses alertes par jour en fonctionnement normal (4 graines, 1 jour simulé par configuration).....
Tableau 41 : Métriques des modèles de référence sur TON_IoT.....
Tableau 42 : Séparation normal/attaque par caractéristique (AUC).....
Tableau 43 : Cadence des lignes, fenêtres normales et d’attaque.....
Tableau 44 : Détection d’anomalies injectées par les modèles de référence.....

Liste des abréviations

ACL : Access Control List

API : Application Programming Interface

AUC : Area Under the Curve (aire sous la courbe ROC)

CA : Certificate Authority (autorité de certification)

CLI : Command Line Interface

CPU : Central Processing Unit

CSV : Comma-Separated Values

DDoS : Distributed Denial of Service

DHCP : Dynamic Host Configuration Protocol

DHT22 : Digital Humidity and Temperature sensor (modèle 22)

DNS : Domain Name System

DoS : Denial of Service

EBIOS RM : Expression des Besoins et Identification des Objectifs de Sécurité – Risk Manager

ENI : Ecole Nationale d'Informatique

ESN : Entreprise de Services Numériques

ESP32 : Espressif Systems 32-bit microcontroller

GNS3 : Graphical Network Simulator-3

GPIO : General Purpose Input/Output

HTTP : HyperText Transfer Protocol

HTTPS : HyperText Transfer Protocol Secure

IA : Intelligence Artificielle

IDS : Intrusion Detection System

IIoT : Industrial Internet of Things

IoT : Internet of Things

IPS : Intrusion Prevention System

ISO : International Organization for Standardization

ITU-T : International Telecommunication Union – Telecommunication Standardization Sector

JPEG : Joint Photographic Experts Group

JSON : JavaScript Object Notation

LAN : Local Area Network

LED : Light Emitting Diode

LMD : Licence-Master-Doctorat

M2 : Master 2

ML : Machine Learning

MQTT : Message Queuing Telemetry Transport

NIST : National Institute of Standards and Technology

NTP : Network Time Protocol

NVS : Non-Volatile Storage (mémoire non volatile de l'ESP32)

OCC : Objets Connectés et Cybersécurité

OS : Operating System

PIR : Passive InfraRed (capteur de mouvement)

QEMU : Quick EMUlator

QoS : Quality of Service (qualité de service MQTT)

RAM : Random Access Memory

REST : Representational State Transfer

RFID : Radio Frequency IDentification

ROC : Receiver Operating Characteristic

SaaS : Software as a Service

SAN : Subject Alternative Name (champ d'un certificat)

SARL : Société à Responsabilité Limitée

SMS : Short Message Service

SMTP : Simple Mail Transfer Protocol

SPI : Serial Peripheral Interface

SQL : Structured Query Language

SSH : Secure SHell

TCP/IP : Transmission Control Protocol / Internet Protocol

TLS : Transport Layer Security

UART : Universal Asynchronous Receiver-Transmitter

UDP : User Datagram Protocol

UI : User Interface

UID : Unique IDentifier (identifiant d'un badge RFID)

UX : User Experience

VLAN : Virtual Local Area Network

VM : Virtual Machine

WAN : Wide Area Network

Wi-Fi : Wireless Fidelity

WSL : Windows Subsystem for Linux

Introduction générale

La généralisation des objets connectés transforme en profondeur l'habitat domestique : ampoules, caméras, thermostats, serrures et capteurs divers communiquent en permanence sur le réseau local pour offrir confort et automatisation. Cette multiplication des points de connexion élargit considérablement la surface d'exposition des foyers aux menaces informatiques, dans un contexte où les utilisateurs disposent rarement des compétences nécessaires pour superviser eux-mêmes la sécurité de leur réseau.

Les mécanismes de sécurité classiques, fondés sur la reconnaissance de signatures d'attaques connues, atteignent rapidement leurs limites face à la diversité des objets connectés domestiques et à l'apparition régulière de nouvelles menaces. Se pose alors la problématique centrale de ce travail : comment détecter, au sein d'un réseau domestique hétérogène, des comportements anormaux d'appareils connectés sans dépendre d'une connaissance préalable de la menace exacte ?

Le présent mémoire a pour objectif de développer le thème suivant : « Mysa : une architecture autonome de cybersécurité comportementale pour les écosystèmes de maisons intelligentes ». Pour y parvenir, l'approche méthodologique adoptée combine une conception architecturale en couches inspirée des modèles de référence de l'IoT, un prototypage matériel et logiciel associant appareils physiques, téléphone recyclé en caméra et appareils simulés, puis une validation empirique au moyen de tests d'intrusion et de scénarios d'anomalie rejoués de façon reproductible, permettant de mesurer la capacité de détection du système et son taux de fausses alertes.

Ce mémoire s'articule en trois parties. La Partie I présente le cadre général du projet : l'établissement pédagogique, l'établissement d'accueil et la description du projet Mysa. La Partie II développe l'analyse préalable, l'architecture IoT retenue et les outils choisis pour la conception. La Partie III est consacrée à la réalisation du prototype, à son installation et à sa configuration, ainsi qu'aux tests de validation permettant d'évaluer la pertinence de l'approche comportementale proposée.

Partie I : PRESENTATIONS

Cette première partie pose le cadre général du projet. Elle présente l'École Nationale d'Informatique, établissement de formation dans lequel s'inscrit ce mémoire, puis UN-IT, l'établissement d'accueil. Elle décrit enfin le projet Mysa : sa problématique, ses objectifs, ses besoins, ses moyens et son planning.

Chapitre 1 : Présentation de l'ENI

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

1.1. Information d'ordre général

L'Ecole Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'Ecole se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'Ecole est la suivante : Ecole Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est la suivante : eni@eni.mg. Il dispose également d'un site web : www.eni.mg

1.2. Missions et historique

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques. Cette Ecole Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI a été créée par le décret N° 83-185 du 24 Mai 1983, comme étant le seul établissement Universitaire Professionnalisé au niveau national, destiné à former des techniciens et des Ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a par conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux notamment :

- En fournissant à des étudiants des connaissances de base en informatique ;
- En leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- En initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC).

La filière de formation d'Analystes Programmeurs a été mise en place à l'Ecole en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'Ecole en 1986.

Une formation de troisième cycle a été ouverte à l'Ecole depuis l'année 2003 – 2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa pour le compte de l'ENI et l'Université Paul Sabatier de Toulouse (UPST).

Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'Informatique, et notamment de préparer la relève des Enseignants-Chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation :

- Génie Logiciel et Base de Données.
- Administration des Systèmes et Réseaux.

La mise en place à l'Ecole de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'Ecole, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mis en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi-présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a été ainsi créé à Fianarantsoa ainsi qu'à l'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouverte au sein de l'Ecole pour répondre aux besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en Mathématiques, en Statistiques ou en Informatique, etc. La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID),
- Objets connectés et Cybersécurité (OCC).

Le principe de l'enseignement pour le parcours GID offre aux étudiants des compétences scientifiques et techniques spécialisées en science des données. Pour le parcours OCC, les étudiants acquièrent une double spécialité, premièrement en internet des objets et deuxièmement en cybersécurité. La formation de master est axée sur l'ensemble des applications de l'Intelligence Artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- Métiers du Digital (MDi) – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale et de la gestion de projets digitaux.
- Audit des Systèmes d'Information (ASI) – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en Mathématiques, en Statistiques ou en Informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne sont organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

À partir de l'année universitaire 2026, dans le cadre de la réorganisation de l'offre de formation, les anciens parcours de la Mention Informatique ont été restructurés et renommés afin de mieux refléter l'évolution des domaines et des compétences visés. Ainsi, l'ancien parcours Génie Logiciel et Base de Données (GB) devient le parcours Ingénierie Logicielle et Systèmes d'Information (ILSI), le parcours Administration des Systèmes et Réseaux (ASR) devient Ingénierie des Systèmes et Réseaux Informatiques (ISRI), tandis que le parcours Informatique Générale (IG) devient Ingénierie des Systèmes d'Information Intégrés (ISII). Cette évolution vise notamment à actualiser les intitulés et à renforcer la cohérence entre les formations proposées et les besoins actuels du secteur numérique.

1.3. Organigramme institutionnel

L'organigramme de l'Ecole est inspiré des dispositions du décret N° 83-185 du 24 Mai 1983. L'ENI est administrée par un Conseil d'Ecole, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants regroupant tous les enseignants-chercheurs permanents de l'Ecole est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché du travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'Ecole Nationale d'Informatique.

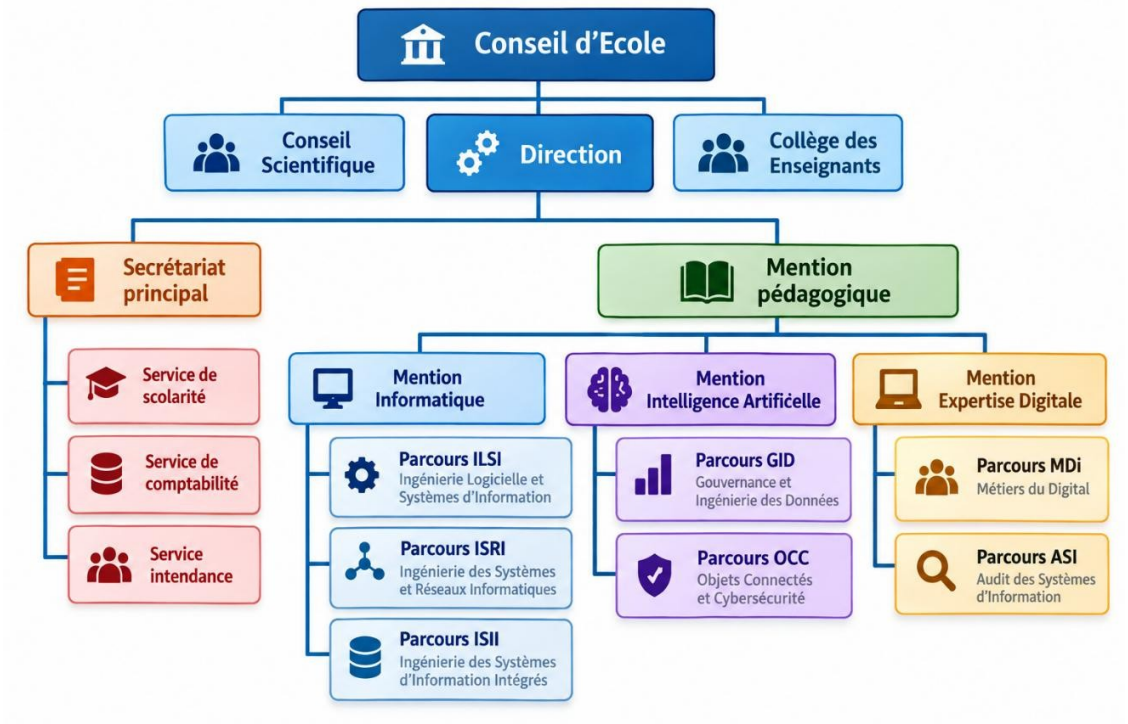


Figure 1 : Organigramme de l'Ecole Nationale d'Informatique

1.4. Domaine de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur les domaines suivants :

- Ingénierie logicielle et systèmes d'information ;
- Ingénierie des systèmes et réseaux informatiques ;
- Ingénierie des systèmes d'information intégrés ;
- Gouvernance et ingénierie des données ;
- Objets connectés et cybersécurité ;
- Métiers du digital et transformation numérique ;
- Audit des systèmes d'information ;
- Intelligence artificielle et ses applications ;
- Modélisation informatique et mathématique des systèmes complexes.

Le tableau 2 décrit l'organisation du système de formation pédagogique de l'Ecole.

Tableau 2 : Organisation du système de formation pédagogique de l'Ecole

Formation théorique	**Formation pratique**
Enseignement théorique Travaux dirigés Travaux pratiques Conférences	Étude de cas Travaux de réalisation Projets / Projets tutorés Voyages d'études

Formation théorique	Formation pratique
	Stages en entreprise

1.5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'Ecole ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe trois mentions et sept parcours. Le tableau 3 récapitule les mentions et les parcours au sein de l'Ecole :

Tableau 3 : Mentions et parcours au sein de l'ENI

Mention	Parcours
Informatique	Ingénierie logicielle et systèmes d'information (ILSI) Ingénierie des systèmes et réseaux informatiques (ISRI) Ingénierie des systèmes d'information intégrés (ISII)
Intelligence Artificielle	Gouvernance et Ingénierie de Données (GID) Objets Connectés et Cybersécurité (OCC)
Expertise Digitale	Métiers du Digital (MDi) Audit des Systèmes d'Information (ASI)

La figure 2 représente l'architecture des études correspondant au système LMD.

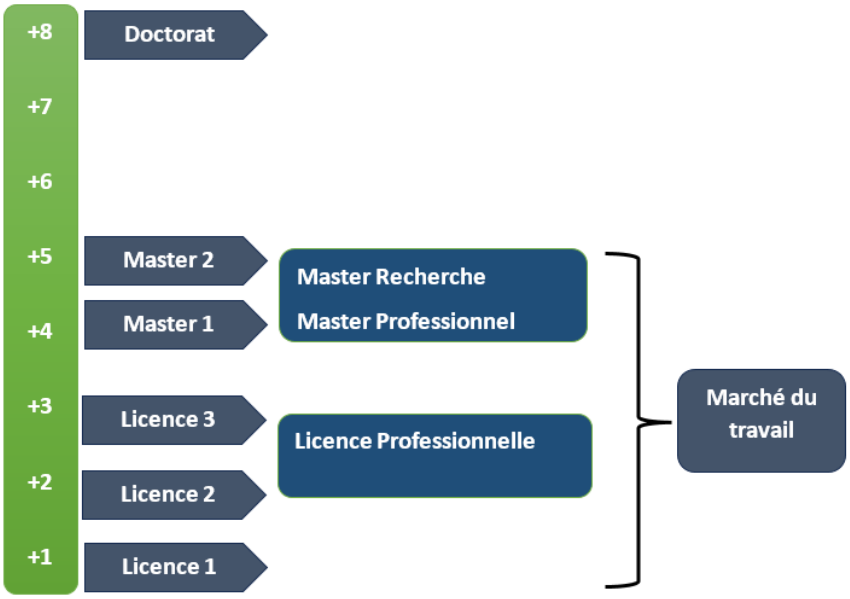


Figure 2 : Architecture des études correspondant au système LMD

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de MASTER se fait automatiquement pour les étudiants de l'Ecole qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 4 illustre la liste des formations existantes à l'ENI.

Tableau 4 : Liste des formations existantes à l'ENI

Formation	Licence professionnelle	Master
Condition d'admission	Par voie de concours	Par voie de concours pour la mention IA
Condition d'accès	Bac de série C, D ou Technique	Être titulaire d'une licence professionnelle
Durée de formation	3 ans	2 ans
Diplôme délivré	Diplôme de Licence Professionnelle	Diplôme de Master Professionnel Diplôme de Master Recherche

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une Ecole Doctorale. Les étudiants diplômés de l'Ecole sont plutôt bien accueillis dans les instituts universitaires étrangers (Canada, Suisse, France, …).

1.6. Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'Ecole en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'Ecole dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes par ces derniers. Les compétences que l'Ecole cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, le travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau MASTER avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'Ecole et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'Ecole, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des Douanes, DLC, E-Tech Consulting, FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB,

MININTER, Ministère des Postes, des Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS, QMM Fort-Dauphin, SG Madagasikara, SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MNDPT, NG ACADEMY, Relia, Spoon Consulting …

1.7. Débouchés professionnels des diplômés

Les formations proposées par l'Ecole permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique et aux TIC.

L'Ecole apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en LICENCE et en MASTER issus de l'ENI peuvent faire carrière dans différents secteurs.

L'Ecole bénéficie aujourd'hui de 40 années d'expériences pédagogiques et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une Ecole Supérieure de référence en matière informatique.

D'une manière générale, les diplômés de l'ENI n'éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l'ENI recommande à ses diplômés de promouvoir l'entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d'études.

Le tableau 5 représente les débouchés éventuels selon la mention.

Tableau 5 : Débouchés éventuels des jeunes diplômés

Mention	**Débouchés professionnels**
INFORMATIQUE	ILSI : Développeur logiciel / développeur full-stack, Ingénieur en développement d'applications (desktop, web, mobile), Architecte logiciel, Intégrateur d'applications, Développeur d'API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer, Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner, Ingénieur QA, Testeur logiciel / testeur automatisé, Spécialiste maintenance applicative, Expert en refactoring et optimisation de code, Data analyst, Concepteur d'outils décisionnels (BI), Ingénieur en Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance ISRI : Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité, Ingénieur cybersécurité, Analyste SOC, Spécialiste firewall / VPN, Responsable sécurité informatique (RSSI – avec expérience), Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation,

Mention	Débouchés professionnels
	Administrateur messagerie (Exchange, Microsoft 365), Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform) ISII : ILSI et ISRI en même temps
INTELLIGENCE ARTIFICIELLE	GID : Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l'Innovation OCC : Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant en cybersécurité, Auditeur en cybersécurité, Expert en sécurité des données, Responsable de la sécurité des informations, Chercheur en cybersécurité, Formateur en cybersécurité, Ingénieur IoT, Développeur IoT, Architecte IoT, Spécialiste en sécurité IoT, Analyste de données
EXPERTISE DIGITALE	MDi : Spécialiste en Marketing Digital, Gestionnaire de Contenu Web, Analyste UX/UI, Product Owner, Chef de Projet Digital, Stratège en Médias Sociaux, Concepteur-Rédacteur Web, Analyste de Performance Digitale, Expert en Commerce Électronique, Spécialiste SEO/SEM, Community Manager, Conseiller en Stratégie Digitale, Responsable CRM, Spécialiste en Business Intelligence, Consultant en Transformation Digitale, Responsable des Partenariats Digitaux, Spécialiste en Cybersécurité Marketing, Directeur de l'Innovation Digitale, Manager de l'E-réputation ASI : Auditeur des Systèmes d'Information, Consultant en Sécurité des Systèmes d'Information, Analyste de Risques IT, Chef de Projet SI, Responsable de la Conformité IT, Spécialiste en Cyber Forensics, Directeur de l'Audit et de la Sécurité des Systèmes d'Information, Expert en Gouvernance IT, Formateur en Sécurité des SI, Chercheur en Sécurité IT

1.8. Ressources humaines

Les ressources humaines sont citées ci-dessous selon leurs responsabilités :

- Directeur de l'Ecole : Monsieur MAHATODY Thomas, Professeur
- Responsable de la Mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences
- Responsable de la Mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences
- Responsable de parcours « ILSI » : Madame RATIANANTITRA Volatiana Marielle, Maître de Conférences
- Responsable de parcours « ISRI » : Monsieur RAZAFINDRAIBE Marolahy Alix, Assistant d'Enseignement Supérieur et de Recherche
- Responsable de parcours « ISII » : Monsieur RAZAFINDRAMONJA Clément Aubert, Assistant d'Enseignement Supérieur et de Recherche
- Responsable de parcours « GID » : Madame RAKOTOASIMBAHOAKA Antsa Cyprienna, Maître de Conférences

- Responsable de parcours « OCC » : Monsieur GILANTE Gesazafy, Assistant d'Enseignement Supérieur et de Recherche

L'ENI compte seize (16) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, neuf (09) Maîtres de Conférences, quatre (04) Assistants d'Enseignement Supérieur et de Recherche, plusieurs enseignants vacataires et personnel administratif.

Chapitre 2 : Présentation de l'établissement d'accueil

Ce chapitre présente UN-IT, l'entreprise de services numériques qui a accueilli la réalisation de ce projet. Il décrit son statut et sa vision, ses activités, ses coordonnées et ses partenaires, puis son organisation interne à travers son organigramme.

2.1. Description et statuts

UN-IT, entreprise de services numériques (ESN), a été fondée en 2022 par un groupe d'ingénieurs passionnés issus de l'ENI et de l'EMIT de l'Université de Fianarantsoa. Leur ambition : transformer les idées en solutions numériques sur mesure tout en stimulant le développement industriel et socioéconomique de Madagascar.

Depuis le 15 mars 2025, UN-IT s'est officiellement constituée en Société à Responsabilité Limitée (SARL), sous la direction de RAFANDROANA Patrice Mahenina et ANDRIANTSIALO Elyah Frisco. Cette étape clé ancre son engagement et structure sa croissance pour mieux servir ses clients et ses ambitions.

Grâce à une expertise technologique reconnue et une approche centrée sur l'humain, UN-IT conçoit des solutions innovantes pour permettre aux entreprises et aux particuliers d'accroître leur visibilité, leur compétitivité et leur rentabilité.

Sa vision : devenir un partenaire de confiance pour les entreprises malgaches et internationales en proposant des solutions numériques de pointe, entièrement adaptées à leurs besoins spécifiques. Convaincue que la transformation digitale est un levier essentiel de croissance, UN-IT s'engage à :

- Fournir des solutions technologiques performantes et durables ;
- Stimuler l'emploi local en accompagnant les jeunes talents dans leur développement ;
- Soutenir les entreprises dans leur transition numérique pour créer un impact positif et durable.

UN-IT, c'est l'alliance entre innovation, expertise et responsabilité sociale, au service de la croissance des entreprises et du développement de Madagascar.

2.2. Activités

UN-IT propose une gamme variée de services pour aider les entreprises et les particuliers à atteindre leurs objectifs ; son activité principale est la création de logiciels personnalisés, d'applications web ou mobiles et de sites web répondant aux besoins de chaque client. Les principales activités et services de UN-IT sont les suivants :

- **Conception UI/UX :** l'utilisateur est placé au centre de la conception, avec des interfaces intuitives et engageantes basées sur des recherches approfondies et des profils utilisateurs détaillés.
- **Développement Web :** des sites e-commerce dynamiques aux blogs personnalisés, des solutions web sur mesure qui répondent aux objectifs commerciaux.
- **Développement Mobile :** des applications multiplateformes optimisées pour iOS et Android, offrant une expérience fluide et cohérente sur tous les appareils.
- **Solutions d'Intelligence Artificielle :** automatisation, personnalisation et optimisation, par l'intégration de l'IA aux processus métiers.
- **Transformation Digitale :** accompagnement stratégique pour optimiser les processus et adopter les dernières technologies.
- **Maintenance Logicielle :** une surveillance proactive pour garantir la performance et la stabilité des logiciels à long terme.
- **Formations :** formations professionnelles pour les entreprises (bureautique, productivité, IA appliquée, outils digitaux) et formations pratiques pour les particuliers (développement web, initiation aux technologies).
- **Solutions SaaS :** solutions logicielles prêtes à l'emploi pour simplifier la gestion des activités.

2.3. Contacts et adresses

Actuellement, le siège de UN-IT se trouve au Lot 0303X/0012B Antamponjina, Fianarantsoa.

- Contact : +261 34 93 035 52
- Courriel : contact@unityfianar.site
- Site web : www.unityfianar.site

2.4. Partenaires

UN-IT collabore avec des partenaires de confiance pour offrir des solutions numériques innovantes et enrichir son impact social et éducatif :

- **ARATO :** spécialiste de la transformation numérique, ARATO soutient le développement de technologies avancées et l'intégration de solutions digitales adaptées aux besoins des entreprises.
- **Kajy University :** institution académique reconnue, Kajy University forme les talents de demain en ingénierie informatique et en technologies émergentes.

- **CERES :** programme éducatif d'excellence créé en 2006 à Fianarantsoa, CERES (Centres de Renforcement Educatif et Scolaire) favorise l'accès à une éducation de qualité pour les collégiens ruraux défavorisés de la région Haute-Matsiatra. En soutenant ces jeunes – dont 50 % de filles – CERES contribue à leur réussite scolaire, leur insertion professionnelle et leur engagement pour le développement de Madagascar.

2.5. Organigramme

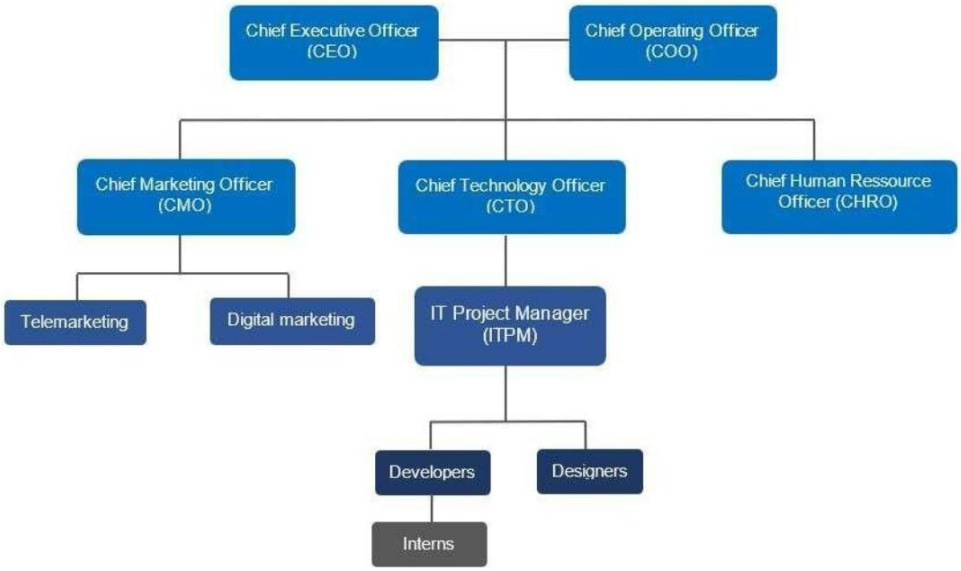


Figure 3 : Organigramme de l'UN-IT

Chapitre 3 : Description du projet

Ce chapitre décrit le projet Mysa. Il formule d'abord la problématique de la sécurité des maisons intelligentes, puis précise l'objectif visé et les besoins des utilisateurs. Il détaille ensuite les moyens nécessaires à la réalisation, les résultats attendus et le chronogramme de travail.

3.1. Formulation

La multiplication des objets connectés au sein des foyers élargit considérablement la surface d'attaque des réseaux domestiques. Caméras, serrures, capteurs et assistants vocaux communiquent en permanence, souvent avec des mécanismes d'authentification faibles, des firmwares rarement mis à jour et une supervision humaine quasi inexistante. Les solutions de sécurité actuellement disponibles pour ce segment reposent en grande majorité sur une logique par signatures : elles comparent le trafic observé à une base de motifs d'attaques déjà répertoriées. Cette approche est par construction incapable de détecter une attaque inédite, une dérive progressive de comportement, ou un usage détourné d'un appareil légitime.

Ce constat conduit à formuler le problème central de ce projet : comment détecter, au sein d'un réseau domestique hétérogène, des comportements anormaux d'appareils connectés sans dépendre d'une connaissance préalable des attaques, tout en restant exploitable par un utilisateur non spécialiste de la cybersécurité ?

Le projet Mysa répond à cette question en proposant une architecture de détection comportementale : chaque flux de télémétrie d'un appareil connecté se voit attribuer un profil d'activité normal, construit par observation continue, et toute déviation significative par rapport à ce profil déclenche une alerte. Cette formulation déplace la question de « quelles attaques connaît-on ? » vers « qu'est-ce qui est normal pour cet appareil ? », ce qui permet en principe de couvrir des menaces non répertoriées. Au fil de la réalisation, cette détection comportementale a été complétée par deux couches explicables, des règles physiques et métier et une corrélation entre capteurs, pour couvrir ce qu'un modèle appris ne sait pas exprimer seul (chapitre 8).

3.2. Objectif et besoins de l'utilisateur

L'objectif général du projet est de concevoir, développer et valider Mysa : une architecture autonome de cybersécurité comportementale pour les écosystèmes de maisons intelligentes. Deux profils d'utilisateurs sont visés : l'occupant, qui veut un état lisible sans jargon, et l'analyste, qui veut le détail des alertes et un historique consultable.

Besoins utilisateurs :

- Affichage de la liste des appareils connectés au réseau domestique, avec leur catégorie et leur état de connexion (en ligne, hors ligne, simulé).
- Supervision en temps réel de l'état de chaque appareil (actif, silencieux, en alerte).
- Détection des anomalies comportementales par rapport à la baseline établie pour chaque flux de télémétrie.
- Calcul et affichage d'un score de risque par appareil, actualisé en continu.
- Génération d'alertes explicables, regroupées en épisodes et acquittables, lors d'un écart comportemental significatif.
- Notification de l'occupant (e-mail, SMS, WhatsApp) lors des événements graves, sans service payant.
- Commande des actionneurs de la maison (serrure, alarme, lampe, badges autorisés) depuis le dashboard.
- Historique consultable des événements et des alertes survenues sur le réseau.
- Journalisation des scénarios d'attaque déclenchés lors des campagnes de validation.

3.3. Moyens nécessaires à la réalisation du projet (humains, matériels, logiciels)

3.3.1. Moyens humains

Le projet est mené par une étudiante en Master 2 Intelligence Artificielle, parcours Objets Connectés et Cybersécurité, encadrée par un encadreur pédagogique de l'ENI pour le suivi méthodologique et par un encadreur professionnel de UN-IT pour l'accompagnement technique.

3.3.2. Moyens matériels

Un ordinateur hôte suffit au développement et à l'exécution de Mysa ; le hub de démonstration tourne sur un poste Windows. Le prototype de maison intelligente comprend deux cartes ESP32 équipées de capteurs et d'actionneurs (PIR, lecteur RFID RC522, servomoteur SG90, buzzer, LED, DHT22, relais), un téléphone Android recyclé servant de caméra, et une maquette de démonstration (entrée et salon). La plateforme d'attaque est une machine virtuelle Kali Linux.

3.3.3. Moyens logiciels

La réalisation de Mysa repose sur un environnement de développement Python pour le moteur de détection comportementale et l'API, Node.js pour le dashboard, PlatformIO pour le firmware, Docker Desktop pour le déploiement du hub, GNS3 avec une appliance OpenWrt

pour la segmentation réseau, ainsi que sur des jeux de données publics pour entraîner les modèles de référence. Un outil de gestion de versions assure la traçabilité du code.

3.4. Résultats attendus

Au terme du projet, les résultats suivants sont attendus :

- Un prototype de maison intelligente opérationnel, combinant appareils physiques, téléphone-caméra et appareils simulés, générant une télémétrie représentative.
- Un moteur Mysa capable d'établir une baseline comportementale stable pour chaque flux du prototype, complété par des règles et une corrélation explicables.
- Un dashboard de supervision fonctionnel, restituant en temps réel l'état et le niveau de risque des appareils.
- Un lab d'attaque automatisé produisant des scénarios reproductibles et journalisés, avec un rapport avant/après.
- Un jeu de résultats quantitatifs (taux de détection, fausses alertes par jour, délai de détection) démontrant l'apport de l'approche comportementale par rapport à une approche par signatures, avec un objectif de moins d'une fausse alerte par jour.

Ces résultats constituent la base empirique du mémoire : ils permettent de discuter, avec des données à l'appui, de la pertinence et des limites de l'approche comportementale appliquée à la sécurité des maisons intelligentes.

3.5. Chronogramme de travail

Le projet est planifié sur une durée de six mois, structurée en huit phases (figure 4). La phase de réalisation s'est achevée par une semaine d'intégration (29 octobre au 5 novembre 2026) consacrée au firmware des deux nœuds, à la corrélation, aux notifications, à la caméra, au déploiement Docker, au réseau GNS3, au lab d'attaque et à la préparation de la démonstration.

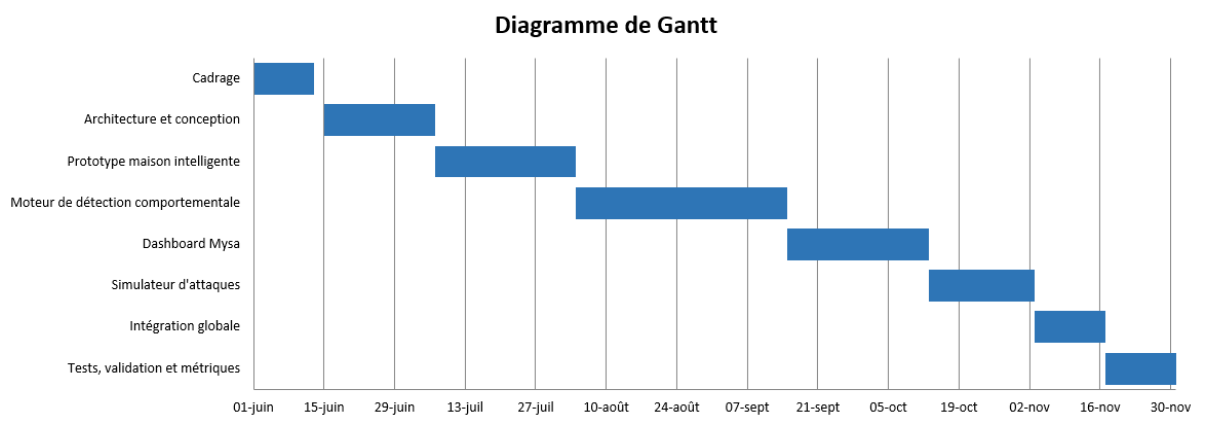


Figure 4 : Chronogramme de travail

PARTIE II : ANALYSE ET CONCEPTION

Cette deuxième partie établit les fondements théoriques et techniques de la solution. Elle analyse l'existant pour en dégager les limites, puis propose et retient une solution. Elle présente ensuite le modèle d'architecture IoT sur lequel repose Mysa. Elle justifie enfin les outils choisis ainsi que la stratégie d'amorçage à froid des modèles de référence.

Chapitre 4 : Analyse préalable

Ce chapitre établit le diagnostic préalable à la conception de Mysa. Il analyse l'existant, c'est-à-dire un prototype de maison intelligente volontairement non sécurisé et les solutions commerciales du marché, en critique les points forts et les points faibles, puis propose deux solutions dont l'une est retenue.

4.1. Analyse de l'existant

L'analyse préalable vise à établir un diagnostic objectif de l'écosystème des maisons intelligentes avant de concevoir une réponse technique. En l'absence d'une infrastructure d'entreprise unique à auditer, cette analyse porte sur l'écosystème type d'un foyer connecté : ses appareils, ses protocoles, ses pratiques de sécurité et les risques qui en découlent. Elle suit une démarche en sept étapes, inspirée des méthodologies d'audit de sécurité et adaptée au contexte d'un réseau domestique et au périmètre du prototype construit dans le cadre de ce projet.

4.1.1. Collecte des données existantes

Cette première étape combine deux sources d'information. D'une part, l'inventaire du prototype construit pour ce projet : un nœud « sécurité et accès » (ESP32 n°1 : capteur PIR HC-SR501, lecteur RFID RC522, servomoteur SG90 actionnant une serrure, buzzer, LED rouge et verte) et un nœud « confort » (ESP32 n°2 : capteur DHT22, module relais commandant une lampe ou une prise), complétés par un téléphone Android servant de caméra, tous reliés par Wi-Fi à un broker MQTT. D'autre part, l'examen de solutions commerciales déjà présentes sur le marché de la sécurité domestique, Firewalla [15], Bitdefender BOX [16] et Fingbox [17], à partir de leur documentation technique et de tests publiés, afin de situer l'approche de Mysa par rapport à l'existant plutôt que de la concevoir dans l'abstrait.

La répartition initiale plaçait le lecteur RFID, la serrure et le relais sur le second nœud. Elle a été revue après les premiers essais : le contrôle d'accès relève de la sécurité, pas du confort, et il a été regroupé sur le nœud sécurité, qui peut ainsi décider localement de l'ouverture de la porte et de l'alarme même sans hub. Le contact de porte à aimant, prévu initialement, n'a finalement pas été câblé ; sa prise en charge reste présente dans le firmware, désactivée.

4.1.2. Analyse des technologies et processus

Du côté du prototype, cette étape examine la chaîne Wi-Fi / MQTT retenue (détaillée en 5.3) : absence par défaut de chiffrement TLS sur le broker Mosquitto en configuration de base, absence d'authentification par appareil tant qu'elle n'est pas explicitement configurée, absence

de protection du firmware ESP32 tant qu'elle n'est pas explicitement activée. Le flux vidéo du téléphone-caméra constitue un second canal, lui aussi exposé s'il n'est ni chiffré ni cloisonné. Du côté des solutions commerciales, l'examen montre des logiques différentes mais toutes réactives : Firewalla et Bitdefender BOX s'appuient sur un filtrage par règles et des scans de vulnérabilités périodiques, de l'ordre de trois jours pour Bitdefender BOX, tandis que Fingbox se limite à la découverte d'appareils et au blocage, sans inspection du contenu des paquets. Aucune des trois ne construit de profil comportemental continu par appareil : c'est précisément l'écart que Mysa cherche à combler.

4.1.3. Recueil des risques et vulnérabilités

Un simple inventaire de risques ne permet pas de prioriser les actions correctives. L'évaluation suit la logique habituelle de l'analyse de risque : une menace exploite une vulnérabilité pour produire un risque, dont la criticité résulte de la combinaison de sa probabilité d'occurrence et de son impact potentiel, selon une grille à trois niveaux inspirée des méthodologies usuelles (EBIOS RM [1], ISO/IEC 27005 [2]).

Tableau 6 : Grille de criticité (impact en lignes, probabilité en colonnes)

Impact \ Probabilité	**Faible**	**Moyenne**	**Élevée**
Faible	Faible	Faible	Moyen
Moyen	Faible	Moyen	Élevé
Élevé	Moyen	Élevé	Critique

Vulnérabilités identifiées sur le prototype non sécurisé :

Tableau 7 : Vulnérabilités identifiées

Réf.	**Vulnérabilité**	**Composant concerné**
V-01	Absence de chiffrement TLS sur les communications MQTT	Broker MQTT
V-02	Identifiants par défaut non modifiés sur le broker	Broker MQTT
V-03	Firmware ESP32 non signé, reprogrammable sans vérification	ESP32 (nœuds)
V-04	Absence de segmentation réseau par défaut	Routeur / passerelle
V-05	Ports de débogage UART exposés physiquement	ESP32 (nœuds)
V-06	Accès physique non contrôlé au routeur	Routeur / passerelle
V-07	Absence de contrôle de la cadence des messages : un compte peut publier sans limite	Broker MQTT, moteur
V-08	Flux vidéo de la caméra accessible depuis tout le réseau local	Téléphone-caméra, hub

Une vulnérabilité ne devient un risque que si une menace est en mesure de l'exploiter. Le tableau 8 associe chaque menace à la ou aux vulnérabilités qu'elle exploite, et évalue le risque qui en résulte :

Tableau 8 : Évaluation des risques

Réf.	Menace	Vulnérabilité exploitée	Probabilité	Impact	Criticité
R-01	Attaquant présent sur le réseau Wi-Fi domestique interceptant le trafic	V-01	Élevée	Élevé	Critique
R-02	Attaquant ayant deviné ou trouvé les identifiants par défaut se connecte au broker	V-02	Élevée	Élevé	Critique
R-03	Attaquant disposant d'un accès temporaire reprogramme ou usurpe un nœud ESP32 et publie de fausses mesures	V-03, V-05	Moyenne	Moyen	Moyen
R-04	Un appareil compromis pivote librement vers le reste du réseau domestique (dont la caméra)	V-04, V-08	Moyenne	Moyen	Moyen
R-05	Intrus ayant un accès physique au domicile manipule directement le routeur	V-06	Faible	Élevé	Moyen
R-06	Un compte détourné inonde le broker de messages (saturation, masquage d'une attaque)	V-07	Moyenne	Moyen	Moyen

Deux risques ressortent en criticité critique (R-01, R-02), et tous les deux exploitent une vulnérabilité du même composant : le broker MQTT, faute de chiffrement et d'identifiants propres. Ce constat oriente directement la priorisation retenue en 4.3 et se retrouve dans la cartographie par composant. Les risques V-07/R-06 et V-08 ont été ajoutés en cours de projet, lorsque la caméra du téléphone a été intégrée et que les scénarios d'attaque ont été écrits.

4.1.4. Cartographie de l'écosystème

Le système existant analysé dans ce chapitre est le prototype de maison intelligente tel qu'il est déployé au départ du projet : des appareils connectés opérationnels, échangeant leur télémétrie, mais sans aucune couche de supervision ni de détection de sécurité. C'est cette configuration volontairement non sécurisée qui sert de point de départ à l'analyse ; la solution destinée à y remédier, Mysa, n'est introduite qu'à partir de la section 4.3.

Dans cette configuration, les appareils connectés (les deux nœuds ESP32, le téléphone-caméra et les appareils simulés) publient leur télémétrie sur un broker MQTT local, accessible depuis l'ensemble du réseau domestique via la passerelle. Cette dernière relie le réseau domestique à Internet, sans segmentation particulière entre les appareils IoT et le reste du réseau. Aucun de ces composants n'est supervisé : une intrusion sur l'un d'eux pourrait se

propager sans être détectée. Chaque composant présente un niveau de criticité différent selon ce qu'il expose en cas de compromission :

Tableau 9 : Cartographie des composants et criticité

Composant	Rôle	Exposition	Criticité
Appareils IoT (ESP32, simulés)	Capteurs / actionneurs, génèrent la télémétrie	Réseau local	Moyenne
Téléphone-caméra	Image de l'entrée, détection de mouvement	Réseau local	Élevée (vie privée)
Broker MQTT (Mosquitto)	Bus de messages central entre tous les appareils	Réseau local	Critique
Passerelle / routeur	Relie le réseau domestique à Internet, aucune segmentation activée	Frontière réseau local / Internet	Élevée
Internet	Vecteur d'exposition externe vers le réseau domestique	Externe	Élevée

Cette lecture par composant converge avec l'évaluation de risques de 4.1.3 : le broker MQTT, identifié comme critique ici comme dans l'analyse de risques, concentre à lui seul deux des risques les plus élevés du projet. Aucun de ces composants ne bénéficiant d'une supervision de sécurité, cette cartographie constitue le socle auquel répond directement la proposition de solutions (4.3).

4.1.5. Sessions de feedback

Des échanges ont été menés avec l'encadreur professionnel du stage ainsi qu'avec un cercle de proches, afin de confronter l'analyse technique à des perspectives extérieures plutôt que de la mener en vase clos. Ces échanges ont porté sur deux aspects distincts. D'une part, ils ont confirmé la pertinence du volet « maison intelligente » du projet dans le contexte malgache : la demande pour ce type d'équipement y est réelle et encore peu couverte par des solutions adaptées au contexte local (coupures d'Internet et de courant, coût des abonnements, préférence pour le SMS). D'autre part, la dimension sécurité du projet ne provient pas de ces échanges mais d'un constat personnel antérieur : la sécurité des maisons intelligentes demeure insuffisamment traitée à l'échelle mondiale, bien au-delà du seul contexte malgache. C'est ce constat qui a orienté le projet vers Mysa plutôt que vers un simple prototype domotique dépourvu de volet cybersécurité.

4.1.6. Rapport d'analyse et recommandations

La mise en commun des constats fait apparaître un écart net entre les trois solutions commerciales examinées et l'approche recherchée dans ce projet : Firewalla et Bitdefender BOX raisonnent par règles et scans périodiques, Fingbox se limite à la présence des appareils,

et aucune des trois ne modélise en continu le comportement normal de chaque appareil. Les recommandations qui en découlent — construire une baseline comportementale par appareil, détecter les écarts sans dépendre d'un cloud tiers, rester lisible pour un utilisateur non expert, fonctionner hors ligne — structurent directement la critique de l'existant (4.2) et la proposition de solutions (4.3).

4.1.7. Mise à jour continue

Le paysage des menaces IoT évolue rapidement : nouveaux types d'appareils, nouveaux protocoles, nouvelles techniques d'attaque. L'analyse présentée ici n'est donc pas figée ; elle a d'ailleurs été réexaminée pendant le projet, lorsque la caméra a été ajoutée (V-08) et que les scénarios d'attaque ont révélé l'absence de contrôle de cadence (V-07). Cette logique fait écho au principe même du moteur Mysa, dont la baseline comportementale est réévaluée en continu plutôt que figée une fois pour toutes (voir 6.2).

4.2. Critique de l'existant (points forts et points faibles)

La critique porte sur deux objets : les solutions commerciales examinées en 4.1.1 et le prototype de départ, volontairement non sécurisé (4.1.4). Le tableau 10 en synthétise les points forts et les points faibles.

Tableau 10 : Points forts et points faibles de l'existant

Élément analysé	Points forts	Points faibles
Firewalla [15]	Solution clé en main, filtrage par règles, utilisable par un non-spécialiste	Raisonnement par règles : aucun profil de comportement continu par appareil
Bitdefender BOX [16]	Scans de vulnérabilités intégrés à une offre grand public	Scans périodiques, de l'ordre de trois jours : fenêtre d'exposition longue ; aucun profil comportemental
Fingbox [17]	Découverte des appareils et blocage, mise en œuvre simple	Pas d'inspection du contenu des paquets ; aucune modélisation du comportement
Prototype de départ (4.1.4)	Chaîne légère Wi-Fi / MQTT adaptée aux ESP32 ; base de comparaison pour mesurer l'apport de chaque mesure	Aucun chiffrement ni identifiants propres, aucune segmentation, aucune supervision (R-01 et R-02 critiques)

Les mécanismes par signatures et par règles, communs à ces offres, détectent des menaces connues mais ne couvrent pas, par construction, les comportements inédits. Aucune de ces solutions ne comble l'écart relevé en 4.1.6, qui consiste à modéliser en continu le comportement normal de chaque appareil. Quant au prototype de départ, il concentre ses risques critiques sur le broker MQTT (4.1.3). Deux besoins en découlent : durcir les composants (chiffrement, authentification, segmentation) et superviser le comportement des appareils. La proposition de solutions (4.3) répond à ces deux besoins.

4.3. Proposition de solutions

Deux approches ont été envisagées pour répondre à la problématique formulée en 3.1. Toutes deux s'appuient sur un même socle de durcissement préventif, issu de l'évaluation des risques (4.1.3) : chiffrement TLS, comptes individuels et listes de contrôle d'accès sur le broker (V-01, V-02), segmentation en VLAN (V-04, V-08). Elles diffèrent par le mécanisme de détection.

Solution 1 : détection par signatures (système de détection d'intrusion classique)

Cette approche repose sur un système de détection d'intrusion s'appuyant sur une base de règles et de signatures d'attaques connues, du type de ceux qui analysent le trafic réseau à la recherche de motifs répertoriés.

- Points forts : solution mature et largement documentée, faible taux de fausses alertes sur les attaques déjà répertoriées, mise en œuvre rapide à l'aide d'outils existants.
- Points faibles : incapacité à détecter des attaques inconnues, maintenance continue de la base de signatures, absence d'adaptation au comportement propre de chaque appareil ; une signature décrit un trafic ou un protocole, et ne permet pas d'exprimer qu'une valeur publiée par un compte légitime est aberrante pour cet appareil.

Solution 2 : détection comportementale par apprentissage de baseline (approche retenue)

Cette approche consiste à construire, pour chaque flux de télémétrie, un profil d'activité normal, puis à signaler tout écart significatif par rapport à ce profil.

- Points forts : capacité à détecter des comportements inédits sans connaissance préalable de la menace, adaptation individuelle à chaque appareil, cohérence avec l'hétérogénéité d'un parc d'objets connectés domestiques, fonctionnement local.
- Points faibles : nécessité d'une phase d'apprentissage avant que la détection ne soit pleinement opérationnelle, risque de fausses alertes, complexité de mise en œuvre supérieure à celle d'une solution par signatures.

Le tableau 11 compare les deux solutions selon les critères déterminants pour un foyer.

Tableau 11 : Comparaison des deux solutions

Critère	Solution 1 : signatures	Solution 2 : comportementale
Menaces inconnues	Non détectées par construction	Détectées si elles modifient le comportement observé
Compte légitime détourné	Non exprimable par une signature	Détecté comme écart par rapport à la baseline
Adaptation à chaque appareil	Règles génériques	Profil propre à chaque flux
Phase d'apprentissage	Aucune	Nécessaire ; couverte par les règles

Critère	Solution 1 : signatures	Solution 2 : comportementale
		explicables et protégée par le filtre de référence (6.2)
Maintenance	Mise à jour continue des signatures	Ré-entraînement du modèle local
Complexité de mise en œuvre	Faible (outils existants)	Plus élevée (modèles, données de référence)

La solution 2 a été retenue. Dans un foyer, la menace principale visée par ce projet n'est pas un trafic reconnaissable mais un compte légitime détourné ou un nœud compromis publiant des valeurs aberrantes, situations qu'une signature ne sait pas décrire. L'hétérogénéité des objets connectés rend en outre difficile le maintien d'une base de signatures exhaustive, alors qu'une approche comportementale s'adapte au profil de chaque appareil et fonctionne localement, comme le recommande 4.1.6.

La réalisation a toutefois montré qu'un modèle appris ne suffit pas seul : il ne sait pas qu'une température de 95 °C est physiquement impossible pour un DHT22, ni qu'un mouvement n'a de valeur d'alerte que si l'alarme est armée et que personne n'a présenté de badge. La solution finale associe donc trois couches : des **règles** physiques et métier explicables, le **modèle comportemental** par flux, et une **corrélation** entre capteurs (chapitre 8). Les règles reprennent l'idée de la solution 1, mais appliquées au contrat de chaque capteur plutôt qu'à des signatures de trafic.

La solution retenue se compose de trois éléments introduits en 3.1 : un prototype de maison intelligente, l'application Mysa (moteur de détection, API, dashboard, notifications) et un lab d'attaque. Le durcissement préventif traite les risques critiques R-01 et R-02 ainsi que R-04, tandis que la détection couvre le risque résiduel de compromission ou d'usurpation d'un nœud (R-03) et la saturation (R-06) ; R-05, lié à l'accès physique au routeur, reste hors du périmètre d'automatisation.

Chapitre 5 : Modèle et architecture de l'IoT

Ce chapitre présente le modèle d'architecture IoT sur lequel repose Mysa. Il rappelle le cadre conceptuel en couches, décrit la couche dispositifs et les technologies de la couche réseau, représente l'architecture appliquée au prototype, la situe parmi les modèles de référence normalisés et en expose les limites.

5.1. Cadre conceptuel et fondements du modèle IoT

Mysa s'organise en quatre couches, inspirées de la recommandation ITU-T Y.2060 [3] : dispositifs, réseau, support aux services et aux applications, application, la sécurité étant traitée comme une capacité transversale.

5.2. Conception et rôle de la couche perception (capteurs et dispositifs)

La couche de perception du prototype regroupe l'ensemble des appareils physiques et virtuels produisant de la télémétrie. Deux nœuds ESP32 physiques assurent la collecte de données réelles :

- Le premier nœud, **esp32-sec-01**, dédié à la sécurité et à l'accès, regroupe un capteur de mouvement PIR HC-SR501, un lecteur RFID RC522, un servomoteur SG90 actionnant une serrure, un buzzer servant de sirène, une LED rouge (mouvement, alarme, badge refusé) et une LED verte (serrure ouverte). L'alarme, le contrôle des badges et la serrure sont gérés localement.
- Le second nœud, **esp32-cmf-01**, dédié au confort, regroupe un capteur DHT22 de température et d'humidité et un module relais commandant une lampe ou une prise, avec un mode thermostat local.

Un téléphone Android recyclé joue le rôle de caméra : il ouvre une simple page web du hub, sans application à installer, et le hub en déduit un capteur de mouvement `home/cam-01/motion`. Ces appareils sont complétés par des appareils simulés (détecteur de mouvement, contact de porte, sonde climatique, ampoule, prise, caméra), générés par script avec une vérité terrain, qui servent à éprouver le moteur de façon reproductible. Chaque appareil est identifié par un `device_id` unique, qui sert à la fois de nom de compte sur le broker et de composant du topic de publication (5.3).

5.3. Technologies de connectivité et protocoles de transmission dans la couche réseau

Au sein de la couche réseau, les nœuds ESP32 se connectent en Wi-Fi au routeur, puis publient leur télémétrie vers un broker MQTT [7] chiffré par TLS sur le port 8883. Le modèle de

publication/abonnement découple les appareils du moteur de détection : celui-ci s'abonne à l'ensemble des topics sans que les appareils aient connaissance de son existence. La comparaison avec CoAP et HTTP, qui justifie le choix de MQTT, est présentée en 6.1.2.

Les appareils publient selon la convention `home/<device_id>/<sensor>`, avec une charge utile au format JSON (`device_id`, `sensor`, `value`, `unit`, `ts`) ; la convention complète figure en 7.1. La confidentialité et l'authentification reposent sur TLS et sur un compte par appareil, l'isolation entre segments sur les VLAN (5.4). La présence des nœuds repose sur le mécanisme de « dernière volonté » (Last Will) de MQTT : à la connexion, chaque nœud déclare un message `offline` que le broker publie lui-même si le nœud disparaît.

Entre le hub et ses clients web, la communication s'appuie sur HTTPS (port 8000) : une API REST pour le dashboard, un flux WebSocket pour les alertes en temps réel, et deux WebSocket pour la caméra (publication par le téléphone, visualisation par le dashboard). HTTPS est imposé par le navigateur, qui n'autorise l'accès à la caméra que sur une origine sécurisée.

5.4. Modélisation schématique d'une architecture IoT appliquée

L'architecture appliquée du prototype est représentée par la figure 5. Elle superpose les quatre couches du modèle retenu et la sécurité transversale.

Architecture en couches de Mysa (modèle ITU-T Y.2060)

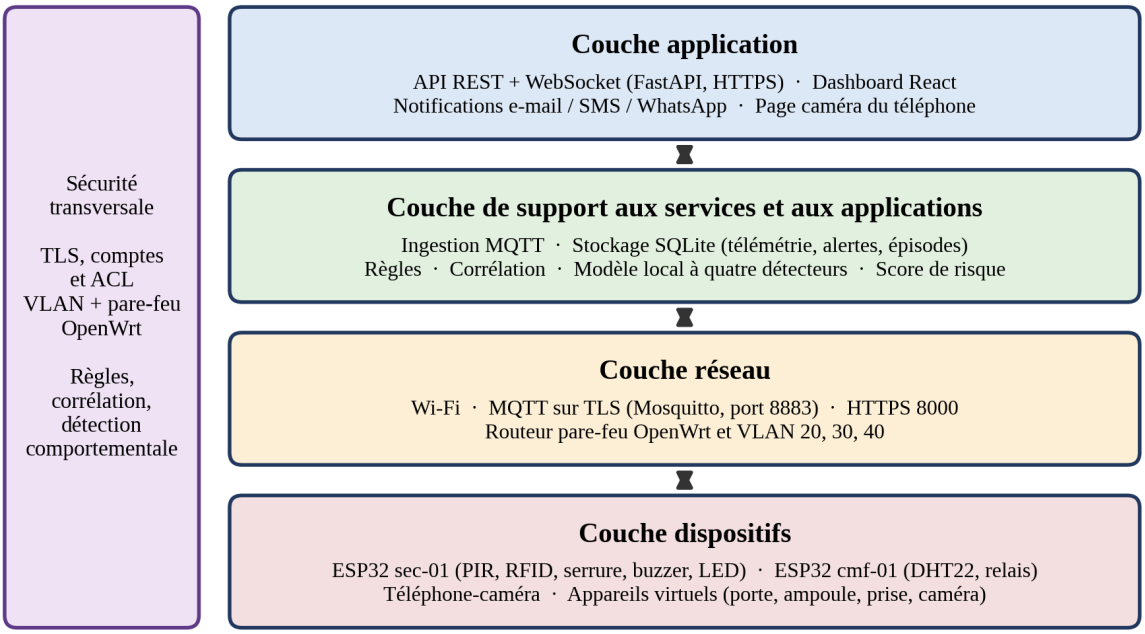


Figure 5 : Architecture en couches de Mysa

La segmentation réseau (figure 6) repose sur trois VLAN, mis en œuvre à l'aide d'un routeur pare-feu OpenWrt simulé sous GNS3 : le VLAN 20 regroupe les objets connectés (nœuds ESP32 et téléphone-caméra), le VLAN 30 le hub qui héberge le broker, le moteur, l'API, le dashboard et la base de données, et le VLAN 40 la plateforme d'attaque. Seuls les flux MQTT chiffrés (8883) et HTTPS (8000) du VLAN 20 vers le hub sont autorisés ; tout autre flux inter-VLAN est refusé, et l'accès du VLAN 40 aux cibles n'est ouvert, règle par règle, que pendant un scénario d'attaque. Le moteur étant lui-même un client du broker, aucune capture du trafic réseau n'est nécessaire à la détection : la segmentation sert à l'isolation, non à l'observation.

La version initiale de la conception prévoyait quatre VLAN (nœuds physiques et appareils simulés séparés) et un routeur MikroTik RouterOS CHR. Elle a été simplifiée : les appareils simulés tournent sur le hub ou sur un poste de test et n'ont pas besoin d'un segment propre, et l'appliance OpenWrt, gratuite et configurable par fichiers texte, a remplacé RouterOS CHR (6.1.7).

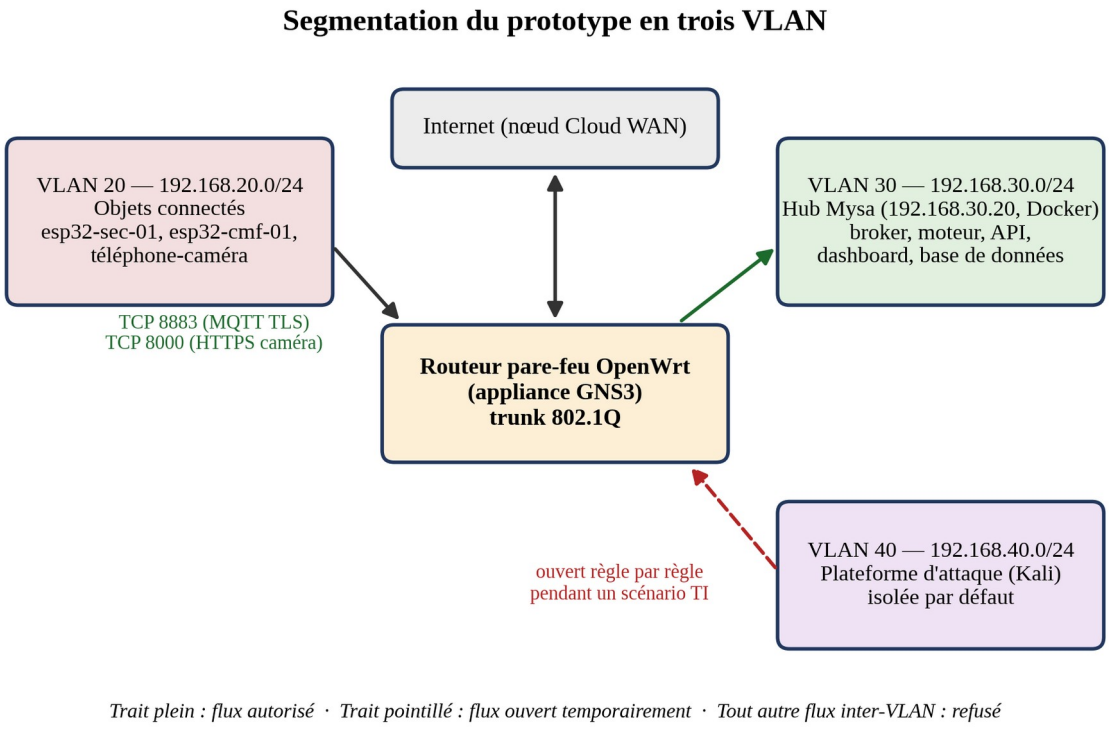


Figure 6 : Segmentation du prototype en trois VLAN

5.5. Analyse comparative des modèles de référence normalisés

Le choix du modèle de référence a une conséquence pratique : il détermine où se place le moteur de détection dans l'architecture. Le tableau 12 compare les modèles examinés sous cet angle.

Tableau 12 : Positionnement de Mysa par rapport aux modèles de référence

Modèle	Ce qu'il apporte	Conséquence pour Mysa
Trois couches : perception, réseau, application [4]	Découpage minimal, très répandu	Ingestion, stockage et détection se retrouvent mêlés à l'interface : le moteur, cœur du projet, n'a pas de place identifiée
ITU-T Y.2060 [3] (retenu)	Quatre couches et capacités de sécurité transversales	Le moteur occupe la couche de support aux services ; la sécurité (TLS, ACL, VLAN, détection) est transversale
oneM2M [5] et ETSI M2M [6]	Plateformes de services communs entre fabricants	Utiles à grande échelle ; hors périmètre d'un prototype à un seul hub

Mysa retient donc la structure en quatre couches de la recommandation Y.2060 : elle donne au moteur de détection une place explicite, et fait de la sécurité comportementale une capacité transversale effectivement mise en œuvre plutôt qu'une couche ajoutée après coup.

5.6. Limites techniques et défis d'ingénierie de l'architecture IoT

Les limites suivantes ont été identifiées au cours de la conception ; pour chacune, la réponse apportée pendant la réalisation est indiquée.

- **Fenêtre d'exposition.** Un modèle local exige un historique d'apprentissage (40 échantillons, soit au moins 215 messages d'un flux). Pendant cette période, la détection comportementale n'est pas active ; elle est couverte par les règles et la corrélation, qui n'ont pas besoin d'apprentissage, et la baseline est protégée par le filtre de référence (6.2).
- **Alertes en grappes.** Le moteur analyse des fenêtres glissantes : une même déviation produit plusieurs alertes consécutives. Les alertes d'un même flux espacées de moins de 60 s sont désormais regroupées en épisodes, unité utilisée par le dashboard, le score de risque, les notifications et les mesures (9.1).
- **Appareil silencieux.** L'arrêt d'un appareil n'était pas détecté, le moteur n'évaluant que les messages reçus. Il est désormais signalé par la présence MQTT (Last Will) pour un nœud entier, et par un chien de garde pour un flux périodique devenu muet (scénario SC-05).
- **Flux événementiels.** Un badge ou une serrure ne publie que lorsqu'il se passe quelque chose : son silence est normal et son rythme ne s'apprend pas. Chaque capteur est donc déclaré périodique, événementiel ou d'état (8.4), et seuls les flux périodiques passent par l'analyse comportementale et le chien de garde.

- **Diversité limitée du parc.** Deux nœuds physiques et un téléphone seulement produisent de la télémétrie réelle ; les appareils simulés et les données publiques compensent partiellement cette limite (6.2).
- **Catégories sans référence.** Le lecteur RFID, le servomoteur et le relais n'ont pas de modèle de référence public et apprennent depuis zéro (6.2.2).

Chapitre 6 : Études et choix des outils

Ce chapitre expose la méthodologie de conception de Mysa et les outils retenus. Il justifie d'abord le choix de chaque technologie, puis la modélisation du moteur de détection et sa stratégie d'amorçage à froid à partir de données publiques, avant de synthétiser l'ensemble des choix.

6.1. Outils utilisés

Le choix de chaque outil résulte d'une mise en balance entre plusieurs alternatives matures, évaluées selon des critères constants : maturité de l'écosystème, courbe d'apprentissage, performance et adéquation avec les contraintes du projet. Cette section détaille cette étude pour chaque brique technologique du projet.

6.1.1. Outils de développement firmware

Tableau 13 : Alternatives pour le firmware ESP32

Alternative	Avantages	Inconvénients
Framework Arduino (retenu)	Simplicité, large bibliothèque de pilotes de capteurs, communauté importante ; FreeRTOS reste accessible	Moins de contrôle bas niveau
ESP-IDF (natif Espressif)	Contrôle fin du matériel, performance optimale	Complexité et courbe d'apprentissage élevées
MicroPython	Prototypage rapide en Python	Moins adapté aux contraintes temps réel des capteurs utilisés

Sources : documentation de PlatformIO [18], d'ESP-IDF [19] et de MicroPython [20].

Le framework Arduino, utilisé via PlatformIO, est retenu : sa simplicité et sa bibliothèque de pilotes (PubSubClient, MFRC522, ESP32Servo, DHT) couvrent largement les besoins du prototype, sans justifier la complexité additionnelle d'ESP-IDF. Les tâches FreeRTOS, accessibles depuis Arduino, ont été utilisées pour que le réseau ne bloque jamais la logique locale (8.3). La version de la plateforme est figée (espressif32 8.0.0, cœur Arduino 3.3.12, mbedTLS 3.6.6) : une version antérieure refusait le certificat du hub.

6.1.2. Protocole et broker de communication IoT

Le tableau 14 compare les trois protocoles envisagés pour la communication entre les appareils et le moteur Mysa.

Tableau 14 : Comparaison de MQTT, CoAP et HTTP/REST

Critère	MQTT	CoAP	HTTP/REST
Modèle de communication	Publication/abonnement via un broker	Requête/réponse (REST)	Requête/réponse (REST)

Critère	MQTT	CoAP	HTTP/REST
Transport	TCP	UDP	TCP
Empreinte protocolaire	Faible (en-tête minimal)	Très faible	Plus élevée (en-têtes textuels)
Fiabilité	Trois niveaux de qualité de service (0, 1, 2)	Messages confirmables ou non	Assurée par TCP, sans qualité de service applicative
Sécurité	TLS	DTLS	TLS (HTTPS)
Découplage appareils / moteur	Fort : le broker s'interpose	Faible : échange direct	Faible : échange direct
Décision pour le projet	Retenu	Écarté : modèle requête/réponse, couplage direct	Écarté pour les capteurs ; retenu pour le dashboard et la caméra

Sources : [4], [7], [8], [9].

MQTT [7] (broker Mosquitto [21]) est retenu face à CoAP [8] et HTTP/REST [9] : son modèle publication/abonnement limite le couplage entre les appareils et le moteur Mysa, son empreinte protocolaire est faible et compatible avec les contraintes des ESP32, et son adoption massive dans l'IoT domestique garantit des bibliothèques matures, tant côté firmware que côté serveur Python (paho-mqtt). Mosquitto offre en natif l'authentification par compte, le chiffrement TLS, le contrôle d'accès par liste (ACL) et la dernière volonté (Last Will), mécanismes directement mobilisés pour traiter V-01 et V-02 et pour suivre la présence des nœuds.

6.1.3. Outils d'apprentissage automatique pour la détection d'anomalies

scikit-learn [11] est retenu pour le moteur comportemental, avec l'algorithme Isolation Forest [10]. Ce choix répond à trois contraintes :

- La détection doit être non supervisée, aucune attaque étiquetée n'étant disponible pour un appareil donné au moment de son installation ;
- Les données sont tabulaires et de faible dimension ;
- Le modèle doit être entraînable sur un nombre réduit d'échantillons et s'exécuter localement sur une machine modeste.

Isolation Forest isole les observations atypiques par partitionnements aléatoires successifs : plus un point est isolé rapidement, plus il est jugé anormal. Il ne se met pas à jour de façon incrémentale : l'adaptation à un appareil passe donc par un ré-entraînement périodique sur une fenêtre glissante d'échantillons récents. La réalisation a montré deux limites de l'algorithme seul : il n'extrapole pas (une valeur très au-delà de la plage d'entraînement n'est pas forcément isolée plus vite) et son seuil, fixé par le paramètre de contamination, est instable sur quelques dizaines de fenêtres très corrélées. Il a donc été entouré de détecteurs déterministes simples

(enveloppes, tendance) et son seuil est calibré sur les scores d'entraînement (8.5). TensorFlow et PyTorch ont été jugés disproportionnés pour ces données tabulaires ; River [22], qui propose des détecteurs incrémentaux, reste une piste pour des travaux futurs.

6.1.4. Framework backend et API

FastAPI [23] est retenu face à Flask, Django et Node/Express : son typage natif réduit les erreurs d'intégration avec le moteur de détection, son support asynchrone convient aux WebSocket (alertes, caméra), sa documentation interactive générée automatiquement (/docs) facilite les tests, et son intégration avec l'écosystème Python scientifique utilisé pour l'analyse comportementale est directe. Servie par uvicorn en HTTPS, l'API distribue aussi le dashboard compilé et la page caméra : un seul service web suffit.

6.1.5. Stockage de la télémétrie

Tableau 15 : Alternatives pour le stockage

Alternative	Avantages	Inconvénients
SQLite (retenu)	Aucun service à installer ni sécuriser, fichier unique, suffisant pour un prototype sur une seule machine	Un seul écrivain à la fois ; ne passe pas à l'échelle d'un déploiement multi-foyers
InfluxDB	Conçu pour les séries temporelles, requêtes temporelles natives	Service supplémentaire à installer, sécuriser et maintenir
PostgreSQL	Robuste, concurrent, extensible (TimescaleDB)	Service lourd pour un prototype à une machine

SQLite [24] est retenu : le hub tourne sur une seule machine, et ajouter un service de base de données élargirait inutilement la surface d'attaque que le projet cherche à réduire. Le moteur et l'API, deux processus distincts, partagent le même fichier ; l'API interroge la base chaque seconde pour diffuser les nouvelles alertes. InfluxDB constitue la piste naturelle si le volume de télémétrie ou le nombre de foyers devait croître.

6.1.6. Framework frontend et visualisation

React [25], associé à Vite pour la compilation, à la bibliothèque Recharts [26] pour les courbes et à une connexion WebSocket pour les mises à jour en temps réel, est retenu face à Vue et Angular pour sa large communauté, la disponibilité de composants de visualisation éprouvés et la réactivité nécessaire à un dashboard de supervision en direct. Grafana, qui permettrait d'obtenir rapidement des tableaux de bord, a été écarté : le dashboard vise deux profils d'utilisateurs (3.2), dont un non spécialiste, ce qui suppose une interface sur mesure (état par pièce, alertes regroupées par épisode, vocabulaire sans jargon, commandes de la maison) plutôt

qu'un outil générique de visualisation de séries temporelles. Les polices sont embarquées dans le dashboard, qui fonctionne ainsi sans accès à Internet.

6.1.7. Environnement de virtualisation, de déploiement et de simulation réseau

Le hub est déployé avec **Docker** [34] (Docker Desktop et WSL2 sous Windows) : trois conteneurs (broker, moteur, API) démarrés par une seule commande, à l'identique sur n'importe quel poste. Ce choix, d'abord envisagé comme une évolution ultérieure, a été avancé lorsque le hub de démonstration a dû passer d'un poste Linux de développement à un poste Windows : il supprime les écarts d'installation entre machines. VMware Workstation [27] reste utilisé pour la machine Kali et pour la GNS3 VM.

La segmentation en VLAN est simulée avec GNS3 [28] et une appliance **OpenWrt** [29] servant de routeur pare-feu. OpenWrt a remplacé MikroTik RouterOS CHR, prévu initialement : l'appliance est gratuite et sans limitation de licence, disponible directement dans la place de marché de GNS3, et sa configuration (réseau, DHCP, pare-feu) tient dans trois fichiers texte versionnés avec le projet. Les ESP32 et le téléphone, équipements Wi-Fi réels, entrent dans la topologie par un nœud Cloud relié à une carte réseau de l'hôte.

6.1.8. Outils de test d'intrusion

Kali Linux [30] est retenu face à des distributions comparables comme Parrot OS : il s'agit de la distribution de référence en test d'intrusion, avec une documentation abondante et un ensemble d'outils préinstallés (nmap, hydra, Wireshark, clients MQTT). Plutôt qu'une machine virtuelle de plusieurs gigaoctets, le lab part de l'image VMware officielle de Kali et un script installe les outils et les scénarios Mysa en une commande. Les scénarios sont écrits en Python (paho-mqtt) pour mesurer automatiquement le résultat de chaque attaque.

6.1.9. Notifications

Trois canaux gratuits, tous facultatifs, ont été retenus pour prévenir l'occupant : l'e-mail par SMTP (Gmail avec mot de passe d'application), le SMS via une passerelle open source installée sur un téléphone Android, qui envoie avec le forfait de sa carte SIM [35], et WhatsApp via le service CallMeBot, gratuit pour un usage personnel [36]. Telegram et les services SMS payants ont été écartés : le SMS fonctionne sans Internet, ce qui correspond au contexte malgache. Le code n'utilise que la bibliothèque standard de Python.

6.1.10. Outils de gestion de projet et de version

Git, avec un dépôt hébergé sur GitHub [31], assure la traçabilité des évolutions du code sur l'ensemble des composants du projet, condition nécessaire pour documenter précisément, dans

le mémoire, l'historique des choix techniques et des itérations successives. Une suite de tests automatisés (pytest) protège le moteur, les règles, la corrélation, l'API et la caméra contre les régressions.

6.2. Données et stratégie d'amorçage à froid

La conception de la solution comporte deux volets. Le volet matériel et logiciel d'ensemble (prototype, blocs de Mysa) est décrit au chapitre 7 et mis en œuvre au chapitre 8. Le volet de modélisation, central pour une approche comportementale, porte sur la manière dont le moteur obtient, pour chaque flux, une référence de comportement normal ; il fait l'objet de la présente section.

6.2.1. Problème : le démarrage à froid

Au branchement d'un appareil, Mysa n'a aucune observation de lui. Un modèle local ne peut être entraîné qu'après avoir réuni suffisamment d'échantillons normaux (5.6), et ces premiers instants ne sont garantis ni représentatifs ni sains : un appareil compromis dès son installation ferait apprendre son comportement anormal comme normal. Cette période est la fenêtre d'exposition.

Pour la réduire, Mysa associe chaque flux à un modèle de référence de sa catégorie, entraîné sur des données publiques, avant de basculer vers un modèle appris sur la télémétrie réelle de l'appareil. Cette démarche s'apparente au transfert de connaissances appliqué au démarrage à froid [12]. La section 6.2.6 et le chapitre 9 montrent que ce modèle de référence, trop grossier pour lever des alertes, a finalement été cantonné à un rôle de filtre de la baseline.

Exemple : le flux `home/esp32-sec-01/pir` est rattaché à la catégorie « mouvement ». Ses premières fenêtres sont évaluées par le modèle entraîné sur IoT_Motion_Light ; celles que la référence juge anormales n'entrent pas dans l'historique d'apprentissage. Après quarante échantillons normaux, un modèle propre au capteur prend le relais.

6.2.2. Datasets publics retenus

TON_IoT (télémétrie IoT). Produit par l'UNSW Canberra, TON_IoT regroupe des données de télémétrie de capteurs IoT et IIoT, des journaux de systèmes d'exploitation et du trafic réseau, collectés sur un banc d'essai mêlant événements normaux et attaques [13][32]. Son usage est libre à des fins de recherche académique, sous réserve de citer les publications des auteurs. Les jeux de télémétrie sont fournis par type d'appareil, avec un horodatage, la ou les valeurs mesurées, une étiquette binaire (normal / attaque) et un type d'attaque. Cette organisation par appareil correspond exactement au besoin : un dataset, une catégorie.

Tableau 16 : Jeux de télémétrie TON_IoT et catégories Mysa associées

Dataset TON_IoT	Grandeurs mesurées	Instances	Catégorie Mysa	Capteur du prototype
IoT_Motion_Light	motion_status, light_status	452 263	mouvement	PIR HC-SR501, caméra (mouvement)
IoT_Garage_Door	door_state, sphone_signal	591 447	ouverture	Contact de porte (prévu, non câblé)
IoT_Weather	temperature, pressure, humidity	650 243	environnemental	DHT22
IoT_Thermostat	current_temperature, thermostat_status	442 229	environnemental (complément possible)	DHT22
IoT_GPS_Tracker	latitude, longitude	595 687	géolocalisation	Aucun (extension)
IoT_Fridge	fridge_temperature, temp_condition	587 077	non retenu	Aucun

Effectifs repris de la description publiée du jeu de données [13].

Aucun jeu ne correspond au lecteur RFID, au servomoteur ni au relais : ces flux n'ont pas de modèle de référence et apprennent depuis zéro.

CICIoT2023. Ce dataset du Canadian Institute for Cybersecurity décrit du trafic réseau d'un parc de 105 appareils IoT soumis à 33 attaques regroupées en sept catégories [14][33]. Il décrit le trafic réseau (flux et paquets) et non la télémétrie applicative d'une catégorie de capteurs. Or, dans sa version actuelle, Mysa analyse la télémétrie MQTT. CICIoT2023 n'est donc pas utilisé pour entraîner les modèles de référence ; il est conservé comme source pour une extension future de la détection au niveau des flux réseau (voir perspectives). Ce positionnement corrige la formulation initiale du projet, qui envisageait d'utiliser les deux datasets de la même manière.

6.2.3. Principe : un modèle par catégorie, jamais de mélange

Le principe retenu comporte trois règles.

- **Un modèle de référence par catégorie, un dataset par modèle.** Chaque modèle de référence est un Isolation Forest entraîné séparément sur le seul dataset de sa catégorie, uniquement sur les fenêtres de télémétrie normales. Les datasets ne sont jamais fusionnés : un modèle entraîné sur l'ensemble produirait un comportement « moyen » de l'IoT, c'est-à-dire exactement le profil unique et peu discriminant que l'approche par appareil cherche à éviter.
- **Association par le nom du capteur.** Lorsqu'un nouveau flux apparaît, Mysa lui associe la catégorie la plus proche d'après le nom de son capteur (6.2.4). Sans correspondance, le flux apprend depuis zéro.

- **Démarrage sous la protection de la référence, puis bascule vers le local et affinage.** Pendant l'amorçage, la référence juge chaque fenêtre complète. Dès que suffisamment d'échantillons normaux de l'appareil sont réunis, un modèle local est entraîné sur ses propres caractéristiques et remplace la référence. Ensuite, le modèle local est ré-entraîné périodiquement sur une fenêtre glissante des échantillons normaux les plus récents.

Tableau 17 : Les trois régimes de fonctionnement d'un flux

Régime	Condition	Modèle qui décide	Apprentissage
1. Amorçage	Pas encore de modèle local ; catégorie reconnue et référence disponible	Référence de la catégorie, en filtre : une fenêtre jugée anormale est exclue de l'apprentissage, sans alerte (par défaut)	Accumulation d'échantillons normaux locaux (un tous les 5 messages)
1 bis. Amorçage sans référence	Catégorie inconnue, ou référence absente ou rejetée par la garde	Aucun modèle comportemental ; seules les règles et la corrélation détectent	Accumulation, comme dans la version initiale
2. Bascule	40 échantillons normaux locaux réunis	Modèle local (nouvellement entraîné)	Entraînement initial
3. Affinage	Modèle local actif	Modèle local à quatre détecteurs	Ré-entraînement tous les 20 nouveaux échantillons sur les 200 plus récents

Isolation Forest n'étant pas incrémental, le transfert ne consiste pas à régler finement un modèle existant : la référence protège la baseline pendant l'amorçage, puis le modèle local, entraîné sur l'appareil, la remplace. L'affinage désigne le ré-entraînement périodique de ce modèle local, qui lui permet de suivre la dérive lente du comportement normal. Le seuil de 40 échantillons (au lieu de 20 dans la version initiale) a été fixé après mesure : avec 20 échantillons, le modèle signalait à tort plus de 15 % des fenêtres d'un flux stationnaire simulé, contre environ 10 % avec 40.

6.2.4. Association capteur–catégorie

Tableau 18 : Association entre nom de capteur et catégorie

Nom de capteur publié	Catégorie associée	Dataset de référence
pir, motion (HC-SR501, caméra)	mouvement	IoT_Motion_Light
magnet, reed, door, contact	ouverture	IoT_Garage_Door
dht22, temperature, humidity	environnemental	IoT_Weather
gps	géolocalisation	IoT_GPS_Tracker (aucun capteur correspondant sur le prototype)
rfid, lock, relay, autre	aucune	Apprentissage depuis zéro

L'association repose sur des mots-clés présents dans le nom du capteur (correspondance exacte d'abord, puis par inclusion). C'est volontairement simple et transparent ; une association fondée

sur le profil statistique du flux plutôt que sur son nom serait une amélioration possible. L'humidité reste rattachée à la catégorie environnementale, bien que cette référence soit entraînée sur des températures : en simple filtre, elle n'a pas causé d'effet indésirable, et la garde de compatibilité l'abandonne si elle ne convient pas au flux.

6.2.5. Caractéristiques portables et caractéristiques locales

Un modèle entraîné sur un dataset ne peut être appliqué à un appareil réel que si les deux sont décrits par les mêmes caractéristiques, calculées de la même façon. Mysa utilise donc une unique fonction de calcul sur fenêtres de 20 messages consécutifs, appliquée aussi bien aux données publiques qu'à la télémétrie réelle. Deux jeux de caractéristiques sont distingués, auxquels s'ajoute une pente de tendance sur une fenêtre plus longue :

Tableau 19 : Caractéristiques d'une fenêtre de 20 messages

Caractéristique	**Définition**	**Portable (référence)**	**Locale**
mean_abs_delta	Variation absolue moyenne entre valeurs consécutives	Oui	Oui
std_value	Écart-type des valeurs	Oui	Oui
unique_ratio	Proportion de valeurs distinctes	Oui	Oui
cv_interval	Coefficient de variation des intervalles entre messages	Non	Oui
mean_value	Valeur moyenne	Non	Oui
mean_interval_s	Intervalle moyen entre messages (s)	Non	Oui
std_interval_s	Écart-type des intervalles (s)	Non	Oui
trend_slope	Pente robuste (Theil-Sen) des valeurs sur les 40 derniers messages	Non	Détecteur de tendance uniquement

Les caractéristiques de rythme et de niveau sont exclues des modèles de référence car elles dépendent de l'appareil et non de la catégorie : une température moyenne mesurée dans un laboratoire australien n'est pas celle d'un salon malgache, et un capteur échantillonné de façon irrégulière ne se comporte pas comme un ESP32 publiant à période fixe. Un essai l'a confirmé : une référence incluant la régularité des intervalles classait comme anormal tout flux périodique. Le prix de cette portabilité est une référence plus grossière, sensible à la dynamique des valeurs (pics, valeurs figées) mais aveugle aux changements de rythme et aux décalages de niveau, que seuls le modèle local et les règles détectent.

6.2.6. Garde-fous et limites

- **Protection contre l'empoisonnement de la baseline.** Une fenêtre jugée anormale, par la référence comme par le modèle local, confirmée ou non, n'est jamais ajoutée à l'historique d'apprentissage. Un appareil compromis dès son installation ne voit ainsi pas son comportement anormal appris comme normal.
- **Garde de compatibilité de la référence.** Si la référence classe en anomalie plus de la moitié des dix premières fenêtres d'un flux, ou des cent dernières (contrôle durable, une référence pouvant se dégrader après un bon début), elle est abandonnée, un avis unique est enregistré et le flux apprend depuis zéro. Cette garde traite le décalage entre le dataset et l'appareil, mais elle ne peut pas le distinguer d'une attaque présente dès le démarrage : sans étiquette, les deux cas sont indiscernables, et l'avis énonce les deux explications.
- **Rôle de filtre plutôt que de détecteur.** Les mesures du chapitre 9 ont montré que la référence détecte les attaques étiquetées de TON_IoT moins souvent qu'elle ne déclenche de fausses alertes, et que ses alertes formaient l'essentiel des fausses alertes du moteur. Par défaut, elle ne lève donc plus d'alerte : elle filtre la baseline pendant l'amorçage. Une variable de configuration rétablit ses alertes pour rejouer l'hypothèse H1.
- **Nature des attaques de TON_IoT.** Les attaques du dataset ciblent la passerelle et les services (dénis de service, injections, mots de passe, etc.) ; leur effet sur les valeurs de télémétrie elles-mêmes peut être faible. Le taux de détection mesuré hors ligne sur ces datasets (9.4) ne doit donc pas être lu comme la performance de Mysa face aux scénarios d'attaque du projet (9.3).
- **Représentativité.** Les conditions de collecte (matériel, localisation, période d'échantillonnage) diffèrent de celles du prototype ; la référence ne peut qu'approcher le comportement normal d'une catégorie.
- **Historique d'apprentissage persistant.** L'historique des échantillons normaux, d'abord conservé en mémoire vive, est désormais sauvegardé sur disque à chaque ajout : un redémarrage du moteur pendant l'apprentissage ne le perd plus.

6.2.7. Protocole d'évaluation

L'apport de l'amorçage n'est pas présumé : il fait l'objet d'une évaluation comparant deux configurations du moteur, apprentissage depuis zéro et référence puis local, sur les mêmes scénarios. Trois hypothèses sont testées :

- **H1.** Lorsqu'une attaque débute pendant la phase d'amorçage, la configuration avec référence en détecte une proportion plus élevée que la configuration depuis zéro, grâce à une baseline non empoisonnée ;
- **H2.** Après la bascule, les performances de la configuration avec référence sont au moins égales à celles de la configuration depuis zéro ;
- **H3.** La référence n'augmente pas le taux de fausses alertes en phase d'amorçage au-delà d'un seuil acceptable, fixé à moins d'une fausse alerte par jour pour l'ensemble du moteur.

Les mesures retenues (taux de détection, fausses alertes, délai de détection, nombre d'épisodes d'alerte) sont définies en 9.1 ; les résultats sont présentés et discutés en 9.3 et 9.5.

6.2.8. Synthèse du choix technologique

Tableau 20 : Synthèse des outils retenus

Catégorie	Outil retenu
Firmware ESP32	Arduino / PlatformIO (espressif32 8.0.0), FreeRTOS
Communication IoT	MQTT (Mosquitto 2) sur TLS, authentification, ACL et Last Will
Détection	Règles explicables, corrélation multi-capteurs, modèle local Python / scikit-learn (Isolation Forest calibré + enveloppes + tendance)
Données de référence	TON_IoT (télémétrie), un modèle par catégorie, en filtre de la baseline ; CICIoT2023 réservé aux perspectives
Backend / API	FastAPI (REST + WebSocket), uvicorn en HTTPS
Stockage	SQLite
Dashboard	React + Vite + Recharts + WebSocket (réalisé)
Caméra	Téléphone Android + page web (HTTPS), détection de mouvement sur le hub (numpy)
Notifications	SMTP, passerelle SMS Android, CallMeBot (WhatsApp)
Déploiement du hub	Docker Compose (Docker Desktop + WSL2 sous Windows)
Segmentation réseau	GNS3 + appliance OpenWrt (VLAN 20, 30, 40)
Lab d'attaque	Kali Linux (VMware) : nmap, hydra, mosquitto-clients, scénarios Python
Tests et gestion de versions	pytest ; Git / GitHub

PARTIE III : RÉALISATION

Cette troisième partie traduit la conception en un prototype fonctionnel. Elle présente l'environnement de réalisation : le prototype de maison intelligente, la plateforme d'attaque et l'architecture de Mysa. Elle détaille ensuite l'installation et la configuration de chaque composant. Elle présente enfin le protocole de test et les résultats de la validation.

Chapitre 7 : Présentation de l'environnement

Ce chapitre présente l'environnement dans lequel Mysa est réalisé : le prototype de maison intelligente, la plateforme d'attaque et l'architecture logicielle de Mysa. Le chapitre 8 en détaille ensuite la mise en œuvre étape par étape, et le chapitre 9 la validation.

7.1. Prototype de maison intelligente

Le prototype combine deux nœuds physiques, un téléphone-caméra et des appareils simulés, reliés au réseau segmenté décrit en 5.4. Chaque appareil possède un identifiant unique, qui sert à la fois d'identité MQTT et de nom de compte sur le broker. Le tableau 21 recense les appareils du prototype. La maquette de démonstration, d'environ 80 × 50 cm, est divisée en deux zones : l'entrée (nœud sécurité) et le salon (nœud confort), le téléphone étant placé sur un support orienté vers l'entrée.

Tableau 21 : Appareils du prototype

Appareil	Type	Capteurs publiés	Actionneurs	VLAN
esp32-sec-01	Physique (ESP32 n°1) — sécurité et accès	pir, rfid (avec UID), rfid_lockout, lock, arm, alarm (avec cause), badges	Serrure (servo SG90), buzzer, LED rouge et verte ; badges autorisés	20
esp32-cmf-01	Physique (ESP32 n°2) — confort	temperature, humidity, dht_error, relay, relay_auto, setpoint	Relais (lampe ou prise), mode et consigne du thermostat	20
cam-01	Téléphone Android (page web) + pont caméra du hub	motion (avec état armé)	—	20
virt-hall-01, virt-door-01, virt-climate-01, virt-bulb-01, virt-plug-01, virt-cam-01	Simulés (script), avec vérité terrain	pir, magnet, temperature, humidity, bulb_state, plug_power, cam_bitrate	—	hub / poste de test

Convention de topics et de messages. Tous les appareils publient selon une convention unique, que le moteur d'ingestion exploite :

- topic de télémétrie : `home/<device_id>/<sensor>` (exemple : `home/esp32-sec-01/pir`) ;
- topic de commande : `home/<device_id>/cmd/<actionneur>`, publié par l'API et ignoré par le moteur d'ingestion ;
- topic de présence : `home/<device_id>/status`, texte `online` publié par le nœud à la connexion, `offline` publié par le broker (Last Will) si le nœud disparaît ;

- topic de simulation : `home/<device_id>/sim` = 1 lorsqu'un appareil est remplacé par le mode simulation de repli (8.11), signalé « Simulé » au dashboard ;
- charge utile au format JSON, avec les champs `device_id`, `sensor`, `value`, `unit` et `ts`, et des champs facultatifs de contexte (`uid` d'un badge, `cause` d'une alarme). Le topic fait foi : un `device_id` de la charge utile qui contredit le topic est rejeté. Le champ `ts` est facultatif ; pour les nœuds physiques, le moteur horodate le message à sa réception.

[Insérer ici la photographie de la maquette avec les deux nœuds ESP32 et le téléphone-caméra (Figure 7)]

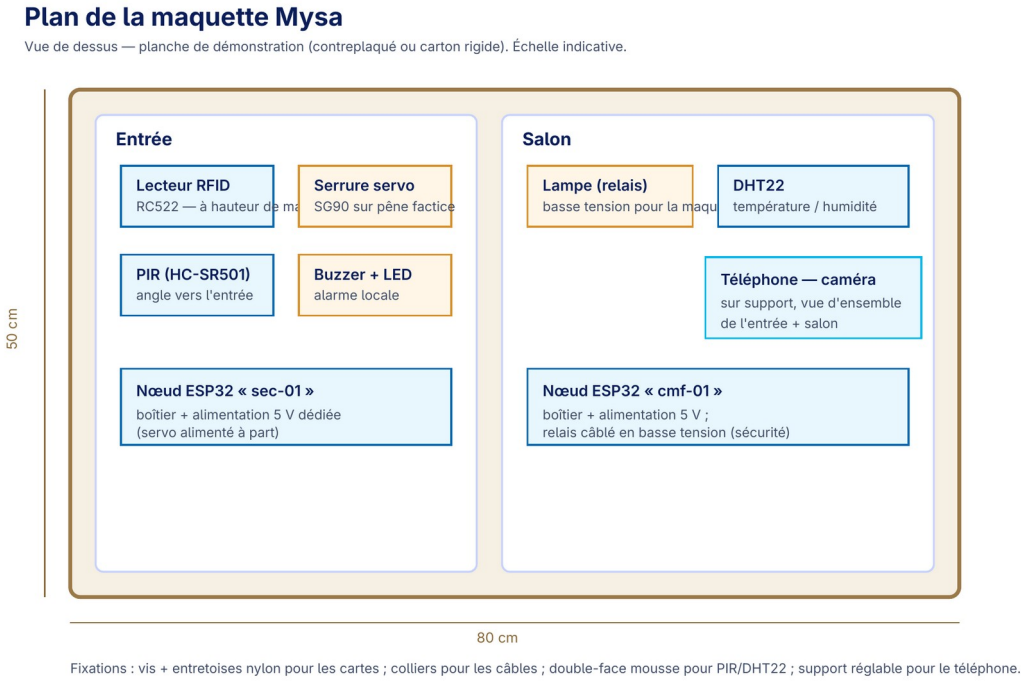


Figure 7 : Plan de la maquette de démonstration (vue de dessus)

7.2. Plateforme d'attaque

La plateforme d'attaque est une machine virtuelle Kali Linux placée dans un VLAN dédié et isolé (VLAN 40). Elle est volontairement séparée du dashboard et du moteur Mysa : aucun code n'est partagé entre les deux, afin que les scénarios éprouvent Mysa comme le ferait un attaquant extérieur, sans connaître son fonctionnement interne. Les scénarios sont regroupés dans le dossier `attack-lab/` du dépôt, chacun relié à un risque évalué au chapitre 4 et à un test d'intrusion (TI-01 à TI-05). Un script les exécute tous et produit un rapport « avant / après ». Les scénarios sont détaillés dans le tableau 36, leur mise en œuvre en 8.12 et leurs résultats au chapitre 9.

7.3. Présentation de Mysa

L'application Mysa regroupe le moteur d'ingestion et de détection, le stockage de la télémétrie (SQLite), l'API (FastAPI), le tableau de bord de supervision (React), le pont caméra et les notifications. Le tableau de bord présente l'état de la maison, les appareils, leur score de risque, les épisodes d'alerte et les commandes des actionneurs. La figure 8 en donne la vue d'ensemble.

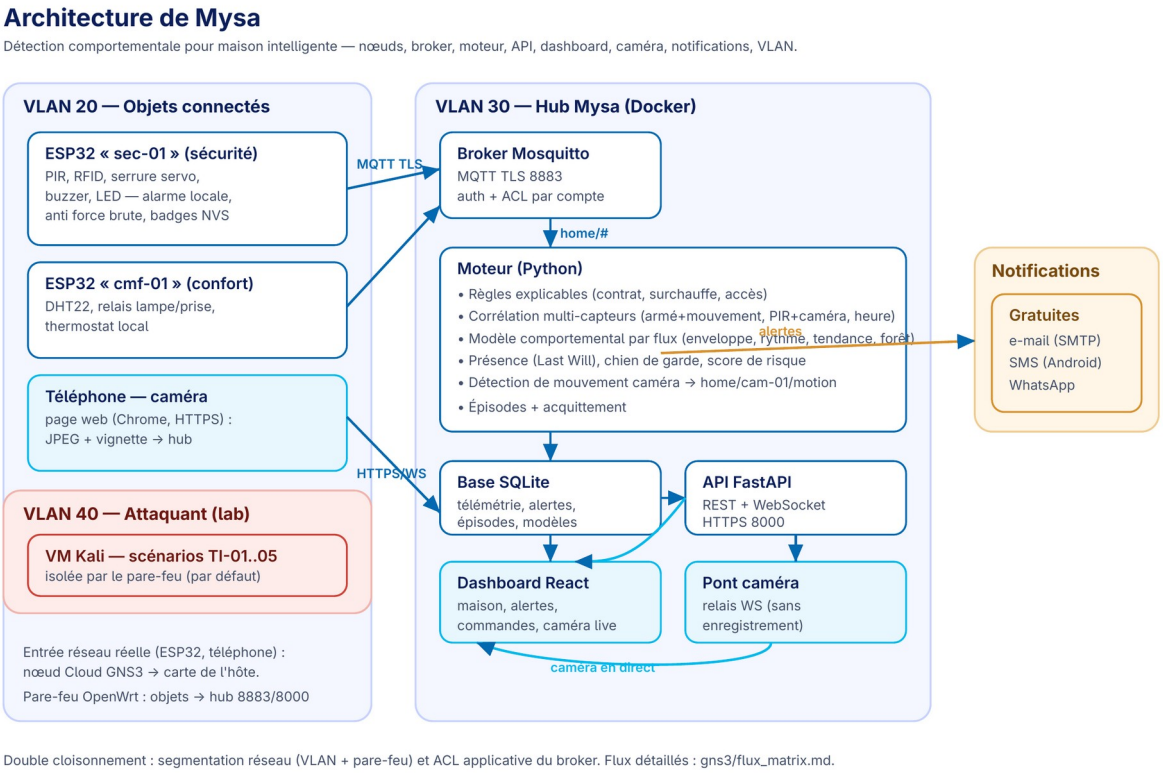


Figure 8 : Architecture d'ensemble de Mysa (nœuds, hub, VLAN, notifications)

Mysa est organisé en blocs successifs, du message MQTT reçu à l'alerte affichée :

Tableau 22 : Blocs logiciels de Mysa et fichiers correspondants

Bloc	Rôle	Fichier(s)
Ingestion	S'abonne à home/#, contrôle le topic, valide et enregistre chaque message, gère la présence, déclenche règles, corrélation et analyse	engine/ingest.py
Stockage	Base SQLite : télémétrie, alertes, épisodes, flux, présence, simulation	engine/storage.py
Profils	Type de chaque capteur : périodique, événementiel ou d'état	engine/profiles.py
Règles	Détections explicables sans apprentissage (contrat du capteur, surchauffe, accès)	engine/rules.py
Corrélation	Détections croisant plusieurs capteurs (C-01 à C-03)	engine/correlation.py
Caractéristiques	Calcule, par flux capteur, les caractéristiques d'une fenêtre de 20 messages et la pente sur 40	engine/features.py
Catégories	Associe un capteur à une catégorie d'appareil	engine/categories.py

Bloc	Rôle	Fichier(s)
Modèle de référence	Charge et applique le modèle de la catégorie (amorçage)	engine/reference.py
Modèle local	Entraîne et applique le modèle à quatre détecteurs propre à un flux	engine/model.py, engine/fastforest.py
Orchestration	Choisit le régime, confirme les indices faibles, lève les alertes	engine/analyzer.py
Chien de garde	Signale un flux périodique devenu silencieux	engine/watchdog.py
Score de risque	Agrège les épisodes récents en un score de 0 à 100 par appareil	engine/risk.py
Vision	Détection de mouvement par comparaison de vignettes de la caméra	engine/vision.py
Notifications	E-mail, SMS, WhatsApp pour les alertes graves et les nœuds hors ligne	engine/notify.py
API	Expose appareils, historiques, épisodes, alertes, commandes, caméra ; sert le dashboard	api/main.py, api/routes/
Dashboard	Interface de supervision et de commande	dashboard/src/
Entraînement des références	Entraîne un modèle de référence à partir d'un CSV public	scripts/train_reference.py
Évaluation	Appareils virtuels, rejeu des scénarios, mesure des fausses alertes	scripts/virtual_devices.py, replay_eval.py, false_alerts.py

L'API expose ces données au dashboard ; ses points d'accès sont récapitulés dans le tableau 23.

Tableau 23 : Points d'accès de l'API

Méthode et chemin	Description
GET /	État du service
GET /api/devices	Appareils vus, dernier message, présence, mode simulé
GET /api/devices/{device_id}/history	Historique de télémétrie (paramètres limit, sensor)
GET /api/devices/{device_id}/risk	Score de risque de l'appareil (0 à 100), niveau et causes
GET /api/streams	État d'apprentissage de chaque flux (régime, référence, échantillons, cadence)
GET /api/episodes	Épisodes d'alerte, les plus récents d'abord (filtre par appareil)
POST /api/episodes/{id}/ack	Acquittement d'un épisode
GET /api/alerts	Alertes brutes (filtres since_id, episode_id)
GET /api/commands ; POST /api/commands	Liste des actionneurs autorisés ; envoi d'une commande validée
GET /api/badges/last-refused	Dernier badge refusé, pour l'enrôler depuis le dashboard
WebSocket /ws/alerts	Flux d'alertes en temps réel pour le dashboard
WebSocket /ws/camera/publish ; /ws/camera/view ; GET /ws/camera/status	Images envoyées par le téléphone ; relais vers le dashboard ; état de la caméra
/dashboard/ ; /camera/	Dashboard compilé ; page caméra du téléphone

La figure 9 représente le cheminement d’un message, de sa publication à son affichage.

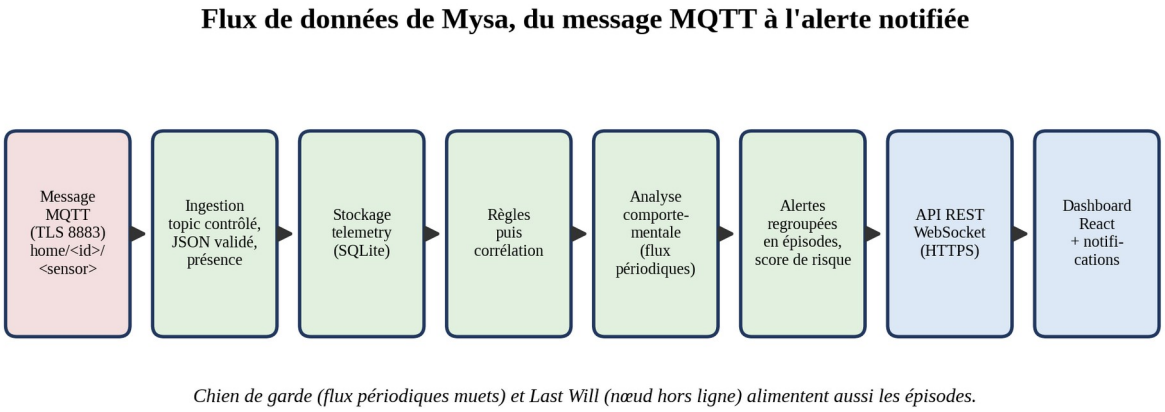


Figure 9 : Flux de données de Mysa

Chapitre 8 : Installation et configuration

Ce chapitre décrit la réalisation étape par étape, dans l'ordre où elle peut être reproduite : d'abord l'infrastructure sécurisée (broker, réseau), puis les nœuds, puis le moteur, les modèles de référence, l'API, le dashboard, la caméra, les notifications, le déploiement et enfin le lab d'attaque. Le tableau 24 en donne la synthèse.

Tableau 24 : État d'avancement de la réalisation

Étape	Section	État
Broker MQTT sécurisé (TLS, comptes, ACL)	8.1	Réalisé et vérifié (TI-01 à TI-03)
Segmentation réseau (OpenWrt, VLAN 20/30/40)	8.2	Configuration et projet GNS3 prêts ; validation sur le poste Windows en cours
Firmware des deux nœuds	8.3	Réalisé, flashé et testé sur les nœuds réels
Ingestion et stockage	8.4	Réalisé
Moteur (règles, corrélation, modèle local, chien de garde, score)	8.5	Réalisé ; 59 tests automatisés au vert
Modèles de référence	8.6	Réalisé (trois catégories)
API	8.7	Réalisé
Dashboard	8.8	Réalisé
Caméra du téléphone	8.9	Réalisé
Notifications	8.10	Réalisé ; comptes à renseigner
Déploiement Docker et mode de repli	8.11	Réalisé ; validé sur le poste Windows
Lab d'attaque TI-01 à TI-05	8.12	Réalisé ; rapport avant/après produit

8.1. Broker MQTT sécurisé

Cette étape répond directement aux vulnérabilités V-01 (absence de chiffrement) et V-02 (identifiants par défaut) du chapitre 4 : le broker n'accepte aucune connexion anonyme et chiffre toutes les communications. Dans le déploiement final, Mosquitto 2 tourne dans un conteneur Docker (8.11) ; la configuration ci-dessous est identique en installation native.

8.1.1. Installer Mosquitto

En déploiement Docker, l'image officielle `eclipse-mosquitto:2` est utilisée et aucune installation n'est nécessaire. En installation native sous Windows, Mosquitto s'installe avec l'installateur de mosquitto.org [21], puis ses dossiers sont ajoutés au PATH de la session.

8.1.2. Créer l'autorité de certification privée et le certificat du hub

Le certificat du hub est signé par une autorité de certification (CA) privée, propre au projet. Le nom d'hôte et l'adresse IP utilisés par les clients pour joindre le hub doivent figurer dans le certificat (champ subjectAltName), sans quoi la vérification échoue. Le même certificat sert au

broker (TLS 8883) et au serveur web du hub (HTTPS 8000), car le navigateur du téléphone impose HTTPS pour l'accès à la caméra. Le script `scripts/gen_certs.sh` (appelé automatiquement par le script de démarrage Windows) produit la CA et le certificat `mysa-hub` :

```
scripts/gen_certs.sh <ADRESSE_DU_HUB> mysa-hub
# broker/certs/ca.crt        autorité de certification (à installer sur le
téléphone)
# broker/certs/ca.key        clé privée de la CA (ne quitte jamais le hub)
# broker/certs/server.crt    certificat du hub (SAN : mysa-hub, localhost, IP du
hub, 127.0.0.1)
# broker/certs/server.key    clé privée du hub
```

Le script repose sur les commandes openssl habituelles (génération d'une clé RSA 2048 bits pour la CA, certificat auto-signé de la CA, demande de signature du serveur, signature avec l'extension subjectAltName). La CA est réutilisée si elle existe déjà ; seul le certificat serveur est régénéré si l'adresse du hub change. Le dossier `broker/certs/` et le fichier `broker/passwd` sont exclus du dépôt (.gitignore) : les clés privées et les mots de passe ne sont jamais publiés.

8.1.3. Créer un compte par appareil

Un compte distinct est créé pour chaque nœud, pour le moteur et pour le pont caméra, de façon à pouvoir restreindre leurs droits individuellement (étape 8.1.4). Le script de démarrage Windows crée ces quatre comptes à partir des mots de passe du fichier `.env` ; en installation manuelle :

```
mosquitto_passwd -c passwd mysa-engine
mosquitto_passwd    passwd mysa-cam
mosquitto_passwd    passwd esp32-sec-01
mosquitto_passwd    passwd esp32-cmf-01
```

8.1.4. Appliquer la configuration et les règles d'accès

La configuration du dépôt (`broker/mosquitto.conf`) désactive les connexions anonymes, n'ouvre que le port chiffré 8883 et référence le fichier de mots de passe et la liste de contrôle d'accès. Le port en clair 1883, un temps ouvert dans une première version Docker, a été supprimé car il réintroduisait V-01.

```
listener 8883
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl.conf
```

Le fichier `broker/acl.conf` applique le principe du moindre privilège : chaque nœud ne peut publier que sous ses propres topics et ne lit que ses propres commandes ; le moteur peut tout lire mais ne publie que des commandes ; le compte caméra est restreint aux topics de la caméra.

```
user esp32-sec-01
topic write home/esp32-sec-01/#
topic read  home/esp32-sec-01/cmd/#

user esp32-cmf-01
topic write home/esp32-cmf-01/#
topic read  home/esp32-cmf-01/cmd/#

user mysa-engine
topic read  home/#
topic write home/+/cmd/#

user mysa-cam
topic write home/cam-01/#
topic read  home/cam-01/cmd/#
```

8.1.5. Vérifier la sécurisation

Tableau 25 : Tests de vérification du broker

Test	Commande (principe)	Résultat attendu	Résultat obtenu
Connexion anonyme	Client MQTT sans identifiants, port 8883	Refus : connexion non autorisée	Refusée (code de retour MQTT 135, « non autorisé »)
Connexion en clair	Client MQTT sans TLS, port 8883	Échec de la poignée de main	Refusée (aucune réponse MQTT)
Force brute	12 mots de passe courants contre mysa-engine	Aucun accepté	Aucun accepté
Connexion authentifiée	Client avec --cafile ca.crt et compte mysa-engine	Connexion établie, messages reçus	Établie ; télémétrie des deux nœuds reçue
Respect des ACL	esp32-sec-01 publie sous home/esp32-cmf-01/...	Message non délivré aux abonnés	Non délivré (usurpation bloquée)

Avec MQTT 3.1.1, un message publié sans autorisation est ignoré par le broker sans erreur renvoyée au publieur : la vérification se fait donc côté abonné, en constatant que le message n'arrive pas. Ces vérifications sont automatisées par les scénarios TI-01 à TI-03 du lab d'attaque (8.12, 9.2).

[Insérer ici la capture des tests de refus et de succès de connexion (Figure 10)]

Figure 10 : Capture des tests de refus et de succès de connexion

8.2. Segmentation réseau en VLAN

La segmentation répond à V-04 (absence de segmentation) et V-08 (caméra exposée) et limite le risque R-04 (pivot depuis un appareil compromis). Elle est réalisée dans GNS3 avec une appliance OpenWrt jouant le rôle de routeur pare-feu entre trois VLAN.

Tableau 26 : Plan de segmentation

VLAN	Rôle	Contenu	Sous-réseau
20	Objets connectés	esp32-sec-01, esp32-cmf-01, téléphone-caméra	192.168.20.0/24
30	Hub Mysa	Broker, moteur, API, dashboard, base de données (hub en 192.168.30.20)	192.168.30.0/24
40	Plateforme d'attaque (isolée)	Machine Kali	192.168.40.0/24
WAN	Internet	Nœud Cloud (DHCP) : notifications, NTP	—

8.2.1. Construire la topologie sous GNS3

Le projet `gns3/mysa.gns3` contient la topologie : le routeur OpenWrt (interface e0 vers un nœud Cloud WAN, interface e1 en trunk 802.1Q vers un commutateur), le commutateur `sw-mysa` (ports d'accès VLAN 20, 30 et 40), un nœud Cloud « objets » relié à une carte réseau de l'hôte, et deux nœuds VMware liés aux machines virtuelles du hub et de Kali. L'appliance OpenWrt s'installe depuis la place de marché de GNS3, et la GNS3 VM fait tourner les appliances.

Les ESP32 et le téléphone ne peuvent pas être branchés dans GNS3 : ils y entrent par le nœud Cloud, relié à la carte réseau de l'hôte qui donne accès au point d'accès Wi-Fi des objets. Le pont vers une carte Wi-Fi fonctionnant mal sous Windows, un adaptateur Ethernet (ou un câble vers la box) est préféré.

8.2.2. Créer les interfaces VLAN sur le routeur

Les fichiers `gns3/openwrt/network`, `dhcp` et `firewall` remplacent la configuration du routeur (`/etc/config/`). Extrait de la configuration réseau, pour le VLAN 30 :

```
config device
    option type '8021q'
    option ifname 'eth1'
    option vid '30'
    option name 'eth1.30'

config interface 'hub'
    option device 'eth1.30'
    option proto 'static'
    option ipaddr '192.168.30.1'
    option netmask '255.255.255.0'
```

Le DHCP distribue un pool par VLAN et réserve l'adresse fixe 192.168.30.20 au hub (adresse MAC à adapter).

8.2.3. Filtrer les flux inter-VLAN

La politique par défaut refuse tout transfert entre zones. Les règles autorisent uniquement les flux nécessaires, résumés dans la matrice de flux du tableau 27. Les règles qui ouvrent le VLAN 40 vers le hub sont présentes dans le fichier, commentées : chaque scénario d'attaque en décommente une seule, ce qui rend les mesures interprétables.

```
config rule
    option name 'Objets-vers-Hub-MQTT'
    option src 'objets'
    option dest 'hub'
    option dest_ip '192.168.30.20'
    option proto 'tcp'
    option dest_port '8883'
    option target 'ACCEPT'
```

Tableau 27 : Matrice de flux du réseau segmenté

Source	Destination	Port / protocole	Autorisé	Justification
Objets (ESP32)	Hub 192.168.30.20	8883/TCP (MQTT TLS)	Oui	Seul canal de télémétrie et de commandes ; chiffré, authentifié, cloisonné par l'ACL
Objets (téléphone)	Hub	8000/TCP (HTTPS)	Oui	Page caméra et WebSocket du flux vidéo
Objets	Hub	Tout autre port	Non	Réduit la surface d'attaque d'un objet compromis
Objets	Internet	Sortant	Oui (optionnel)	NTP / DNS ; peut être retiré pour un cloisonnement strict
Hub	Internet	Sortant	Oui	Notifications (SMTP, CallMeBot) et mises à jour
Hub	Objets	Réponses MQTT	Oui (suivi de connexion)	Les commandes passent dans la connexion ouverte par l'objet
Attaquant	Hub / objets	—	Non par défaut	Isolé ; ouvert règle par règle selon le scénario TI joué
Hub (administration)	Routeur	22, 80, 443	Oui	Administration du pare-feu depuis le seul segment du hub

8.2.4. Vérifier l'isolation

Depuis un objet du VLAN 20 : les ports 8883 et 8000 du hub répondent, tout autre port est inaccessible. Depuis le VLAN 40 (Kali) : aucun accès aux VLAN 20 et 30 hors scénario d'attaque. Ces vérifications sont à exécuter sur le poste Windows qui héberge GNS3 ; leurs résultats seront reportés ici.

[Insérer ici la capture de la topologie GNS3 avec les trois VLAN et le résultat des tests d'isolation (Figure 11)]

Figure 11 : Topologie GNS3 avec les trois VLAN

8.3. Firmware des nœuds ESP32

Le projet PlatformIO (`firmware/`) contient deux firmwares de production, `sec01` et `cmf01`, et deux programmes de test matériel sans réseau (`hwtest_sec`, `hwtest_cmf`) qui vérifient chaque composant isolément avant le flashage définitif. Les identifiants Wi-Fi et MQTT et le certificat de la CA sont regroupés dans `include/secrets.h`, exclu du dépôt.

```
[env]
platform = espressif32@8.0.0      ; cœur Arduino 3.3.12 / mbedTLS 3.6.6
board = esp32dev
framework = arduino
monitor_speed = 115200

[env:sec01]
build_src_filter = +<sec01/>
lib_deps = knolleary/PubSubClient @ ^2.8
           miguelbalboa/MFRC522 @ ^1.4.11
           madhephaestus/ESP32Servo @ ^3.0.5

[env:cmf01]
upload_speed = 115200             ; 921600 fait échouer le flashage sur cette
carte
build_src_filter = +<cmf01/>
lib_deps = knolleary/PubSubClient @ ^2.8
           adafruit/DHT sensor library @ ^1.4.6
```

8.3.1. Couche réseau commune

Les deux nœuds partagent une couche réseau (`include/mysa_net.h`) conçue pour que la logique locale ne soit jamais bloquée par le réseau. Dans une première version, une reconnexion MQTT bloquante empêchait la serrure de se refermer et le lecteur RFID de lire lorsque le broker était coupé. La version finale fait tourner le Wi-Fi et MQTT dans une tâche FreeRTOS dédiée :

- les publications sont déposées dans une file, vidée par la tâche réseau dès que la connexion est établie ;
- les commandes reçues sont placées dans une seconde file, lue par la boucle principale ;
- le nœud déclare à la connexion une dernière volonté retenue `home/<id>/status = offline` et publie `online` ;
- la session MQTT est persistante et l'abonnement aux commandes en QoS 1 : une commande envoyée pendant une coupure est délivrée à la reconnexion ;
- la connexion TLS vérifie le certificat du hub avec la CA privée (`setCACert`).

8.3.2. Nœud sécurité et accès (esp32-sec-01)

Toute la logique de sécurité est locale : la serrure, le contrôle des badges et l'alarme fonctionnent même si le hub ou le Wi-Fi sont coupés ; le hub supervise et commande.

- **Badges :** la liste des UID autorisés est conservée en mémoire non volatile (NVS) et modifiable à distance (commandes `badge_add` et `badge_del`). Un badge autorisé ouvre la serrure 5 s (LED verte allumée) ; un badge refusé allume la LED rouge et publie `rfid = 0` avec son UID.
- **Protection contre la force brute :** après 3 badges refusés en 60 s, le lecteur est bloqué 30 s et `rfid_lockout = 1` est publié.
- **Alarme :** une fois armée, après un délai de sortie de 30 s, un mouvement sans badge valide dans les 60 s qui précèdent déclenche la sirène (60 s au plus) et publie `alarm = 1` avec sa cause.
- **Mouvement :** le PIR est publié toutes les 2 s ; un mouvement bref entre deux publications est mémorisé pour ne pas être perdu.

Tableau 28 : Brochage du nœud esp32-sec-01

Composant	Broche ESP32	Remarque
PIR HC-SR501 (OUT)	GPIO 13	Alimenté en 5 V
Lecteur RFID RC522	SDA 5, SCK 18, MISO 19, MOSI 23, RST 22	Alimenté en 3,3 V (bus SPI)
Servomoteur SG90 (signal)	GPIO 27	Alimentation 5 V séparée, masse commune
Buzzer actif	GPIO 25	Sirène et bips
LED rouge + 220 Ω	GPIO 26	Mouvement, alarme, badge refusé
LED verte + 220 Ω	GPIO 21	Serrure ouverte
Contact de porte MC-38	GPIO 14	Abandonné ; fonction désactivée dans le firmware

8.3.3. Nœud confort (esp32-cmf-01)

Le nœud confort publie la température et l'humidité toutes les 5 s (le DHT22 ne supporte pas une cadence inférieure à 2 s) et signale une lecture impossible (`dht_error`). Le relais, actif à l'état bas, a deux modes conservés en NVS : manuel (commandé depuis le dashboard) et automatique, où un thermostat local allume le relais au-dessus de la consigne (28 °C par défaut) et l'éteint en dessous de la consigne moins 0,5 °C, sans dépendre du hub. Le DHT22 est relié au GPIO 4 et le relais au GPIO 27 ; le relais commande une lampe basse tension sur la maquette, pour des raisons de sécurité électrique.

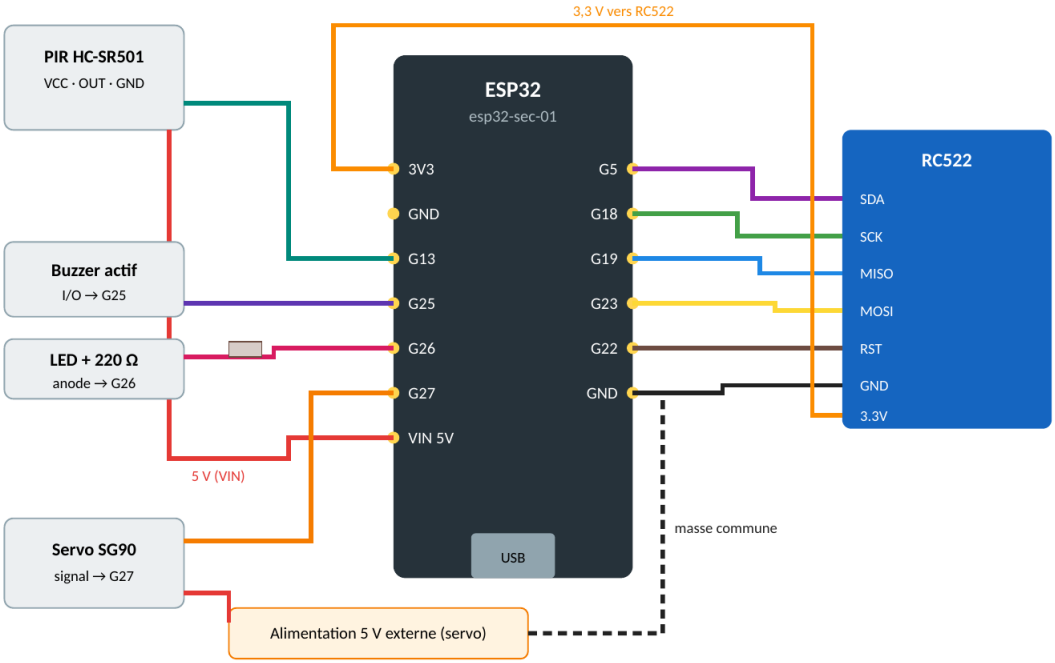


Figure 12 : Schéma de câblage du nœud esp32-sec-01 (la LED verte, sur GPIO 21, n'est pas représentée)

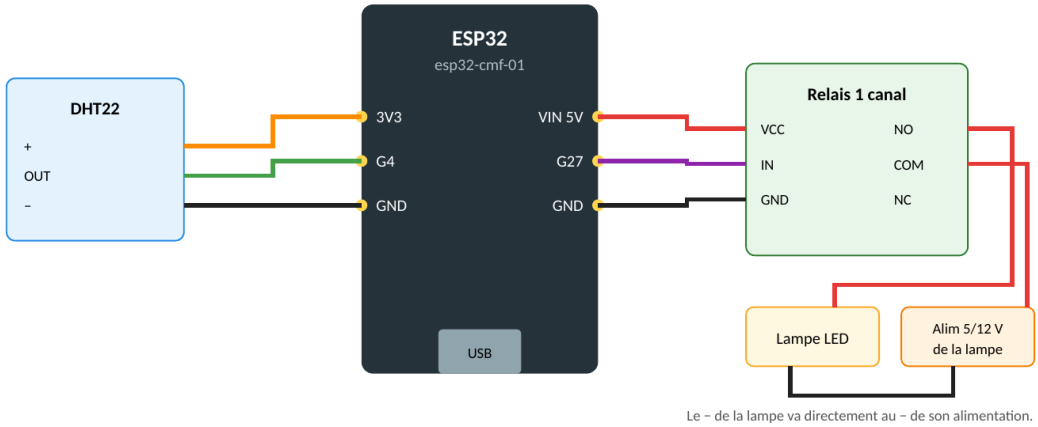


Figure 13 : Schéma de câblage du nœud esp32-cmf-01

8.4. Ingestion MQTT et stockage

8.4.1. Préparer l'environnement Python

En déploiement Docker (8.11), l'environnement est construit automatiquement. En développement :

```
python -m venv .venv
.venv\Scripts\activate              (Linux : source .venv/bin/activate)
pip install -r requirements.txt
Copy-Item .env.example .env          # puis renseigner les valeurs
```

Le fichier `.env` (non versionné) contient l'adresse du broker, le compte du moteur, l'activation de TLS, le chemin du certificat de la CA (`MQTT_CA_CERT`), le chemin de la base (`DB_PATH`),

l'adresse fixe du hub (`MYSA_HUB_IP`), les mots de passe des comptes et les paramètres des notifications.

8.4.2. Le schéma de stockage

Six tables sont créées automatiquement au démarrage, avec des index sur l'appareil, le capteur et l'horodatage :

Tableau 29 : Tables SQLite

Table	Colonnes principales	Rôle
telemetry	id, device_id, sensor, value (réel), unit, ts	Télémétrie brute
alerts	id, device_id, sensor, message, severity, kind, source, score, episode_id, ts	Alertes brutes
episodes	id, device_id, sensor, kind, source, severity, start_ts, end_ts, alert_count, min_score, acquittement	Alertes regroupées (même flux et même source, moins de 60 s d'écart)
streams	device_id, sensor, category, regime, ref_status, local_samples, last_ts, ema_interval, message_count, silent	État d'apprentissage et cadence de chaque flux
presence	device_id, online, ts	Nœud en ligne ou hors ligne (Last Will)
simulated	device_id, simulated, ts	Appareil remplacé par le mode simulation de repli

8.4.3. Lancer le moteur d'ingestion

Le moteur s'abonne à `home/#` avec le compte `mysa-engine`, ignore les topics de commande, traite les topics de présence et de simulation, rejette les messages non JSON, non numériques ou dont le `device_id` contredit le topic, enregistre chaque télémétrie valide, puis la soumet successivement aux règles, à la corrélation et, pour les flux périodiques seulement, à l'analyse comportementale.

```
python -m engine.ingest
```

Chaque capteur est déclaré dans `engine/profiles.py` selon trois profils : **périodique** (PIR, température, humidité, relais : analyse comportementale et chien de garde), **événementiel** (badge, serrure, alarme, armement, mouvement de la caméra : seules les règles s'appliquent, le silence est normal) ou **d'état** (nombre de badges, consigne, mode du relais : enregistré pour le dashboard, jamais analysé). Ce découpage corrige un défaut de la première version, où le chien de garde aurait signalé comme anormale une heure sans passage de badge.

8.5. Moteur de détection

Le moteur combine trois couches complémentaires. Les règles et la corrélation sont explicables et actives dès le premier message ; le modèle comportemental repère ce qui s'écarte de l'habitude d'un flux, une fois cette habitude apprise.

8.5.1. Règles explicables

Les règles ne demandent aucun apprentissage. Elles couvrent ce que le modèle ne sait pas (ce qui est physiquement impossible) et les événements d'accès, qui n'ont pas de rythme à apprendre.

Tableau 30 : Règles du moteur

Règle	Condition	Gravité
Valeur hors contrat	Capteur binaire recevant autre chose que 0 ou 1 ; température hors [-40 ; 80] °C ou humidité hors [0 ; 100] % (message forgé ou capteur défaillant)	high
Surchauffe	Température ≥ 45 °C	high
Variation brutale	Température variant de plus de 5 °C en moins de 60 s	medium
Badge refusé	Badge non autorisé présenté	low
Accès refusés répétés	3 badges refusés en moins de 60 s	high
Lecteur bloqué	Blocage du lecteur après plusieurs refus (force brute)	high
Alarme	Alarme déclenchée par le nœud, avec sa cause	high
Capteur en défaut	Lecture du DHT22 impossible	medium
Mouvement filmé	Mouvement détecté par la caméra alors que l'alarme est armée	high

8.5.2. Corrélation entre capteurs

La corrélation reconnaît des scénarios de sécurité qu'aucun capteur ne révèle seul. Sans état propre, elle relit tout depuis la base : un redémarrage du moteur ne lui fait rien perdre.

Tableau 31 : Corrélations entre capteurs

Réf.	Condition	Interprétation	Gravité
C-01	Mouvement (PIR ou caméra) alors que l'alarme est armée, hors délai de sortie de 60 s, sans badge valide dans les 120 s précédentes	Intrusion probable	high
C-02	Mouvement confirmé par le PIR et par la caméra à moins de 30 s d'intervalle, alarme armée	Présence confirmée par deux capteurs indépendants	high
C-03	Badge accepté à une heure vue dans au plus 5 % des accès des 30 derniers jours (ou entre 0 h et 5 h tant que moins de 20 accès sont connus)	Accès à une heure inhabituelle	medium

8.5.3. Caractéristiques et modèle local à quatre détecteurs

Pour chaque nouveau message d'un flux périodique, le moteur relit les 20 derniers messages du même capteur du même appareil (et 40 pour la tendance). Tant que la fenêtre n'est pas pleine, aucune évaluation n'a lieu. Une fonction unique calcule les caractéristiques du tableau 19, aussi bien pour la télémétrie réelle que pour les datasets publics.

Le modèle local, entraîné par flux, ne repose plus sur le seul Isolation Forest. La première version, réglée par le seul paramètre de contamination, produisait l'essentiel des fausses alertes (scores à peine sous le seuil) et laissait passer des valeurs très au-delà de la plage d'apprentissage, car la forêt n'extrapole pas. Le modèle combine désormais quatre détecteurs, chacun avec un rôle précis :

Tableau 32 : Détecteurs du modèle local

Détecteur	Ce qu'il repère	Principe	Force de l'indice
Enveloppe de valeurs	Pic, valeur hors domaine (SC-01)	Une caractéristique de valeurs sort de la plage d'entraînement de plus de 5 fois son étendue	Fort : alerte immédiate (high)
Enveloppe de rythme	Inondation, ralentissement (SC-02, TI-04)	Intervalle moyen hors de [min / 3 ; max × 3] des valeurs d'entraînement	Fort
Tendance	Dérive progressive (SC-03), capteurs continus	Pente de Theil-Sen sur 40 messages plus de 2 fois supérieure à la plus forte pente d'entraînement	Fort
Bascules	Capteur binaire qui oscille	Taux de changement d'état plus de 2 fois supérieur au maximum d'entraînement (au moins 0,5)	Fort
Isolation Forest calibré	Combinaison inhabituelle de caractéristiques	Score inférieur au plus bas score d'entraînement, d'une marge de 0,5 fois l'écart (médiane − minimum)	Faible : confirmé si 2 fenêtres sur 3 sont suspectes (medium)

L'Isolation Forest (100 arbres, contamination de 1 %, graine fixe) est entraîné sur les sept caractéristiques locales et sauvegardé dans `engine/models/<appareil>__<capteur>.joblib` afin de survivre à un redémarrage. L'évaluation d'un seul échantillon par scikit-learn coûtant environ 30 ms, les arbres entraînés sont « compilés » en tableaux numpy et parcourus directement (`engine/fastforest.py`) : le résultat est identique, vérifié par un test automatisé, et l'évaluation est environ cent fois plus rapide.

8.5.4. Orchestration des régimes

Pour chaque flux, `analyzer.evaluate` choisit le modèle qui décide (local s'il existe, sinon référence de la catégorie en filtre, sinon aucun), lève une alerte pour un indice fort, confirme

les indices faibles, et alimente l'historique d'apprentissage uniquement avec des fenêtres normales, à raison d'un échantillon tous les 5 messages. Après la reconnexion d'un nœud, ses fenêtres contiennent forcément le trou de la coupure : pendant une période de grâce de 120 s, le détecteur de rythme ne lève pas d'alerte, la coupure étant déjà signalée par la présence.

Tableau 33 : Paramètres du moteur

Paramètre	Valeur	Rôle
WINDOW / TREND_WINDOW	20 / 40 messages	Taille de la fenêtre d'analyse / de la fenêtre de tendance
STRIDE	5 messages	Un échantillon d'apprentissage tous les 5 messages
N_LOCAL_MIN	40 échantillons	Seuil d'entraînement du premier modèle local (≥ 215 messages)
N_LOCAL_MAX	200 échantillons	Taille de la fenêtre glissante d'apprentissage
REFIT_EVERY	20 échantillons	Fréquence de ré-entraînement (affinage)
CONFIRM_K / CONFIRM_M	2 / 3	Confirmation des indices faibles
REF_CHECK_WINDOWS / REF_ROLLING_WINDOWS	10 / 100 fenêtres	Fenêtres observées pour juger la référence, au démarrage puis en continu
REF_MAX_FLAG_RATE	0,5	Part d'anomalies au-delà de laquelle la référence est rejetée
MYSA_REFERENCE_ALERTS	0 (désactivé)	La référence filtre la baseline sans lever d'alerte
RECONNECT_GRACE_S	120 s	Grâce du détecteur de rythme après reconnexion d'un nœud
EPISODE_GAP_S	60 s	Écart maximal entre deux alertes d'un même épisode

Ces valeurs ont été ajustées à l'aide du banc d'essai hors ligne (9.3), en visant moins d'une fausse alerte par jour sans perte de détection.

8.5.5. Présence et chien de garde

Un nœud déclaré hors ligne par le broker (Last Will) lève une alerte « hors ligne » et une notification. Pour un flux périodique, le chien de garde compare l'âge du dernier message au rythme habituel du flux (moyenne mobile des intervalles) : au-delà de max(10 s, 10 × intervalle habituel), après au moins 10 messages, le flux est déclaré silencieux. Un flux événementiel n'est jamais surveillé ainsi, et les flux d'un nœud déjà hors ligne ne sont pas signalés une seconde fois.

8.5.6. Vérification de bout en bout

Avec un broker de développement et le moteur lancé, le script de simulation publie 150 messages normaux d'un capteur PIR puis une valeur aberrante :

```
python scripts/simulate_device.py
```

Le terminal du moteur affiche la progression de l'apprentissage puis une alerte, également consultable via `GET /api/alerts` et sur le dashboard. La même chaîne a été vérifiée avec les nœuds réels le 29 octobre 2026 : nœud ESP32 → broker Mosquitto (TLS 8883, authentification) → moteur → API → dashboard, avec badge autorisé accepté, badge inconnu refusé, commandes reçues et alerte de silence levée au débranchement d'un nœud.

8.5.7. Score de risque par appareil

Le score de risque résume l'état d'un appareil en un nombre de 0 à 100, lisible par un non-spécialiste. Il est calculé à partir des épisodes d'alerte récents de l'appareil : chaque épisode contribue selon sa gravité (low 10, medium 20, high 40), et sa contribution décroît avec l'âge de sa dernière alerte (constante de 30 minutes), de sorte qu'un appareil redevenu normal voit son score revenir vers zéro. Les épisodes, plutôt que les alertes brutes, sont comptés afin que les fenêtres glissantes ne gonflent pas artificiellement le score. Un flux actuellement silencieux ajoute 30 points et un nœud actuellement hors ligne 40 points, sans décroissance, tant que la situation dure.

```
# engine/risk.py (extrait)
WEIGHTS = {"low": 10, "medium": 20, "high": 40}
SILENT_WEIGHT, OFFLINE_WEIGHT = 30, 40
TAU_S = 30 * 60

def risk_score(episodes):
    """episodes : liste de (gravité, âge en secondes) ; retourne un entier de 0 à
100."""
    total = sum(WEIGHTS.get(sev, 10) * math.exp(-max(age, 0.0) / TAU_S) for sev,
age in episodes)
    return min(100, round(total))
```

Trois niveaux sont affichés au dashboard : « Tout va bien » (0 à 24), « À surveiller » (25 à 59) et « Alerte » (60 à 100). Le score est exposé par `GET /api/devices/{device_id}/risk`, avec la liste des causes qui le composent.

8.6. Modèles de référence et amorçage à froid

8.6.1. Télécharger les datasets

Les jeux de télémétrie TON_IoT sont téléchargés depuis la page officielle de l'UNSW (6.2.2) dans le dossier `data/public/`, exclu du dépôt (volume et conditions d'usage). Avant tout entraînement, les noms de colonnes et les valeurs réelles sont vérifiés dans chaque fichier : le script convertit les valeurs textuelles (par exemple ouvert/fermé) en codes numériques.

8.6.2. Entraîner un modèle par catégorie

```
python scripts/train_reference.py --category mouvement --csv
data/public/IoT_Motion_Light.csv
python scripts/train_reference.py --category ouverture --csv
data/public/IoT_Garage_Door.csv
python scripts/train_reference.py --category environnemental --csv
data/public/IoT_Weather.csv
```

Pour chaque catégorie, le script découpe la série en fenêtres de 20 lignes non chevauchantes, calcule les caractéristiques portables, entraîne l'Isolation Forest sur 80 % des fenêtres entièrement normales, puis évalue le modèle sur les 20 % restants (fausses alertes) et sur les fenêtres majoritairement composées d'attaques (détection). Il écrit le modèle et un fichier de métriques dans `engine/models/reference/`. Un second script (`scripts/eval_injection.py`) mesure la sensibilité de chaque référence à des anomalies injectées (pic, valeur bloquée, oscillation, décalage de niveau).

8.6.3. Contrôler le résultat

Les métriques de chaque modèle sont reportées aux tableaux 41 et 44. Elles ont conduit à cantonner la référence au rôle de filtre de la baseline (6.2.6).

8.6.4. Démarrage d'un nouvel appareil

Lorsqu'un nouvel appareil publie, `categories.categorize` associe chaque capteur à une catégorie (temperature → environnemental, pir → mouvement) ; le modèle de référence correspondant est chargé s'il existe, et juge chaque fenêtre complète de 20 messages pendant l'amorçage. Un capteur sans catégorie (RFID, relais) suit le régime d'amorçage sans référence. L'état de chaque flux (régime, statut de la référence, nombre d'échantillons) est enregistré dans la table `streams` et affiché sur la page « Moteur » du dashboard.

8.7. API de consultation et de commande

```
uvicorn api.main:app --reload                        # développement
curl -k https://<hub>:8000/api/devices
curl -k https://<hub>:8000/api/episodes
curl -k "https://<hub>:8000/api/devices/esp32-sec-01/history?limit=50"
```

L'API lit la même base que le moteur ; la documentation interactive est disponible sur /docs. Le flux WebSocket `/ws/alerts` interroge la base chaque seconde et envoie au dashboard les nouvelles alertes ; la déconnexion du client termine proprement la boucle. En déploiement, l'API est servie en HTTPS avec le certificat du hub et distribue aussi le dashboard compilé (`/dashboard/`) et la page caméra (`/camera/`).

Les commandes des actionneurs sont publiées sur `home/<device_id>/cmd/<actionneur>` avec le compte du moteur, seul autorisé par l'ACL à écrire sur ces topics. Seuls les actionneurs et valeurs déclarés sont acceptés (serrure, alarme, sirène, buzzer, voyant, ajout et retrait d'un badge au format UID, relais, mode et consigne du thermostat entre 10 et 40 °C) : l'API ne sert pas de relais MQTT générique. L'API n'ayant pas d'authentification propre, elle doit rester dans le VLAN du hub ; une variable permet de désactiver les commandes.

[Insérer ici la capture de la documentation interactive /docs (Figure 14)]

Figure 14 : Capture de la documentation interactive de l'API (/docs)

8.8. Dashboard de supervision

Le dashboard (React, Vite, Recharts) consomme l'API et le flux WebSocket. Il vise deux profils (3.2) : l'occupant, qui veut un état lisible sans jargon, et l'analyste, qui veut le détail des alertes et un historique consultable.

8.8.1. Initialiser et compiler le projet

```
cd dashboard
npm install
npm run dev          # développement : http://localhost:5173 (relaie /api et /ws
vers l'API)
npm run build        # production : dashboard servi par l'API sur /dashboard/
```

8.8.2. Vues réalisées

Tableau 34 : Pages du dashboard

Page	Pour qui	Contenu
Maison	L'occupant	État de la maison sous le toit du logo (bleu, ambre ou rouge), une « pièce » par appareil avec ses dernières mesures et son score de risque, panneau de commandes (serrure, alarme, lampe, thermostat, badges), caméra en direct, journal des épisodes en phrases simples
Alertes	L'analyste	Épisodes filtrables (anomalies, règles, silences, avis, appareil), détecteur, gravité, durée ; chaque épisode se déplie sur ses alertes brutes ; acquittement
Appareil	Les deux	Score de risque et ses causes, une courbe par capteur avec les épisodes surlignés, étape d'apprentissage, commandes des actionneurs
Moteur	Le jury, l'analyste	Étape d'apprentissage de chaque flux, référence, cadence, et ce qui déclenche une alerte

8.8.3. Principes d'interface

- Statuts lisibles : « Tout va bien », « À surveiller », « Alerte » plutôt que des valeurs brutes, pour le profil occupant.

- Alertes regroupées par épisode, le détail (valeur, source de décision, fenêtre) étant réservé à la vue analyste ; un épisode traité est acquitté depuis le dashboard.
- Une notification apparaît à chaque nouvel épisode reçu par WebSocket.
- Couleur toujours doublée d'un libellé, pour rester lisible sans distinction de couleur.
- Badge « Simulé » sur un appareil remplacé par le mode de repli, et état en ligne / hors ligne de chaque nœud.
- Enrôlement d'un badge depuis le dernier badge refusé, sans reflasher le nœud.
- Polices embarquées : aucune connexion Internet n'est nécessaire.

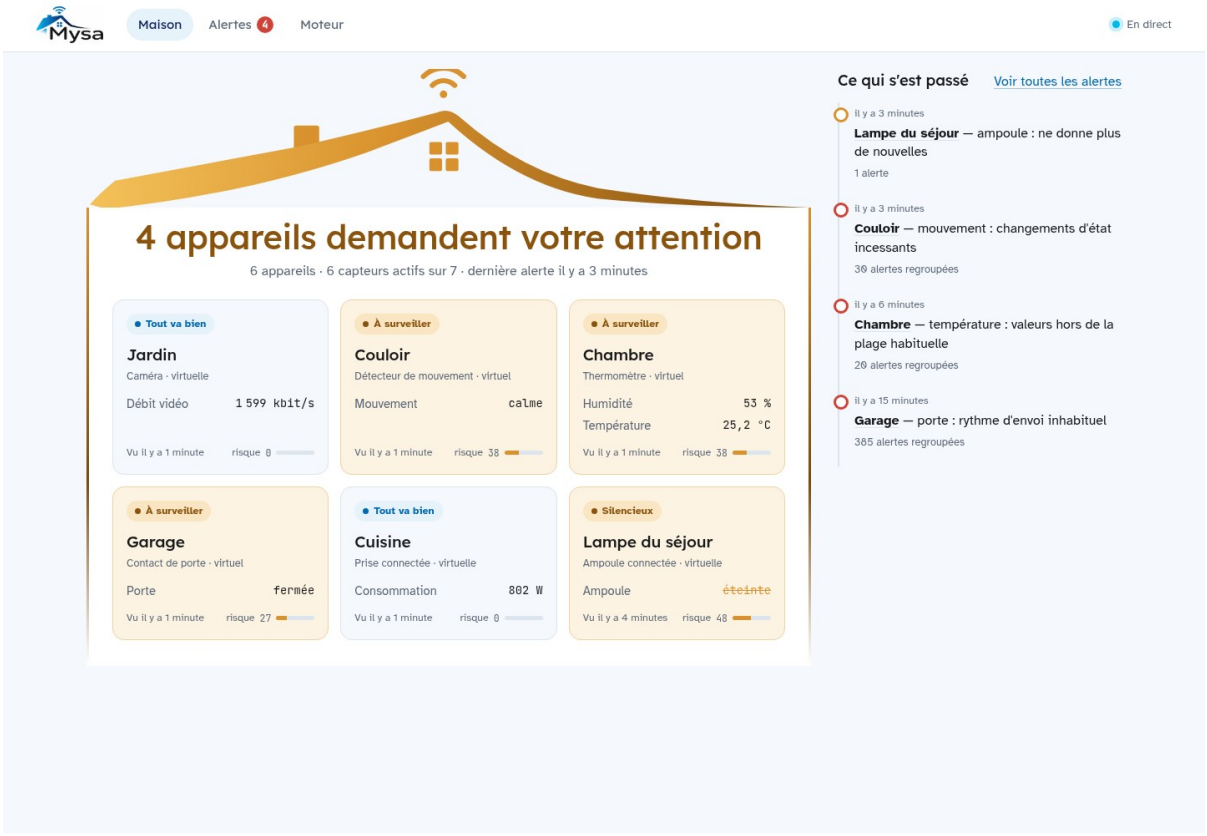


Figure 15 : Dashboard — page « Maison » (état de la maison, pièces, score de risque)

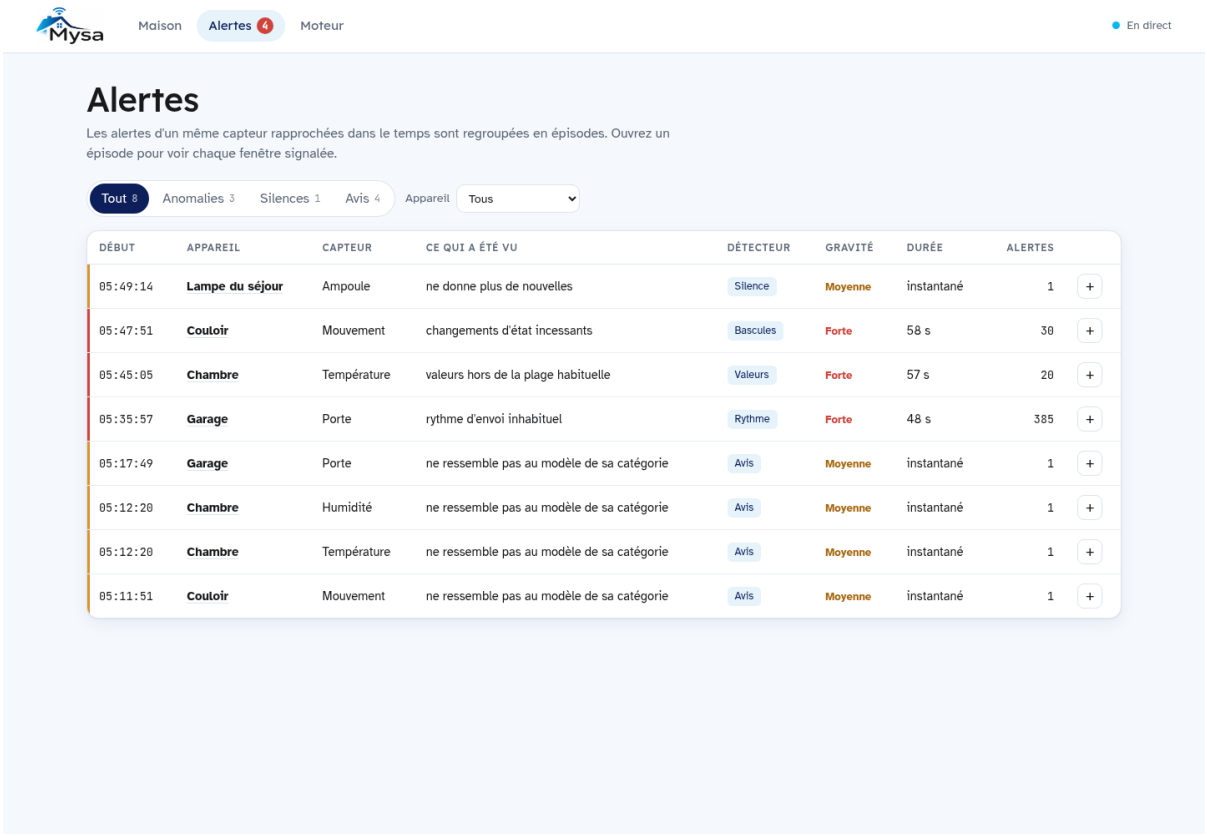


Figure 16 : Dashboard — page « Alertes » (épisodes filtrables, gravité, acquittement)

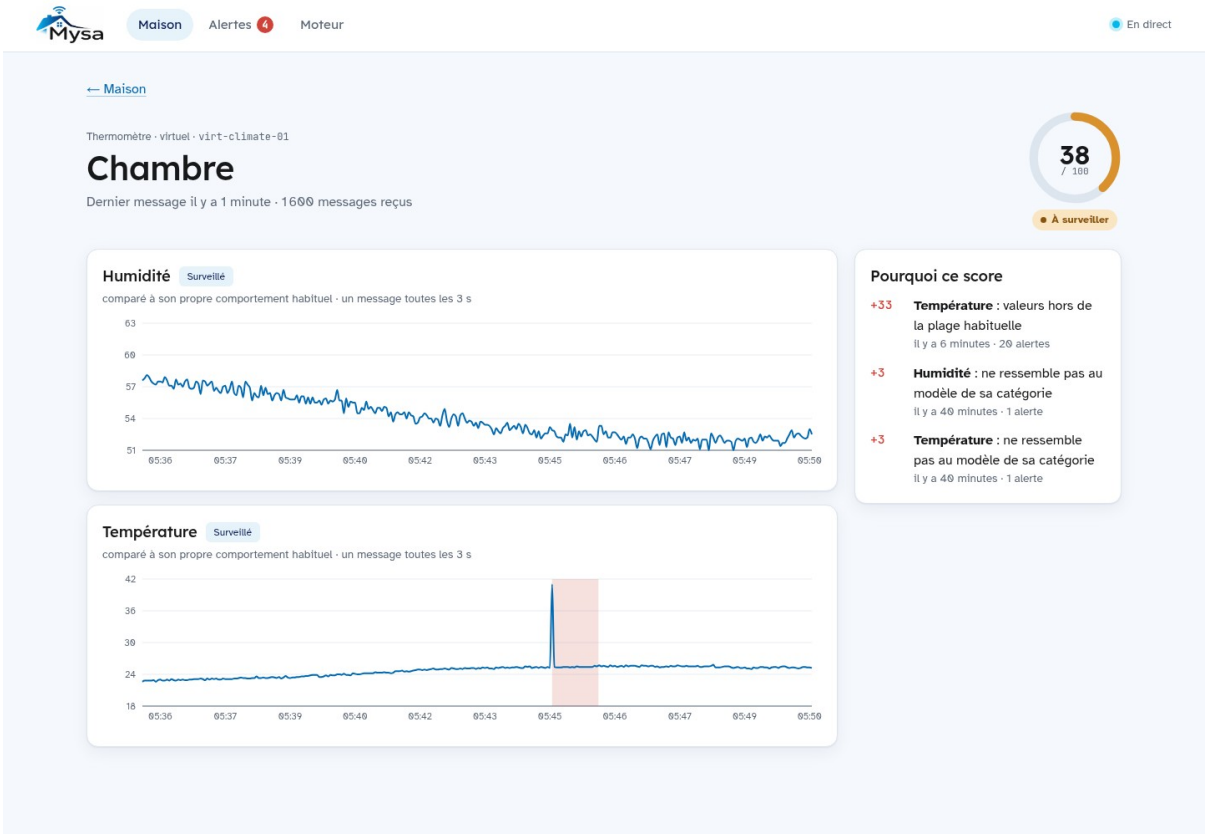


Figure 17 : Dashboard — détail d'un appareil (courbes, causes du score)

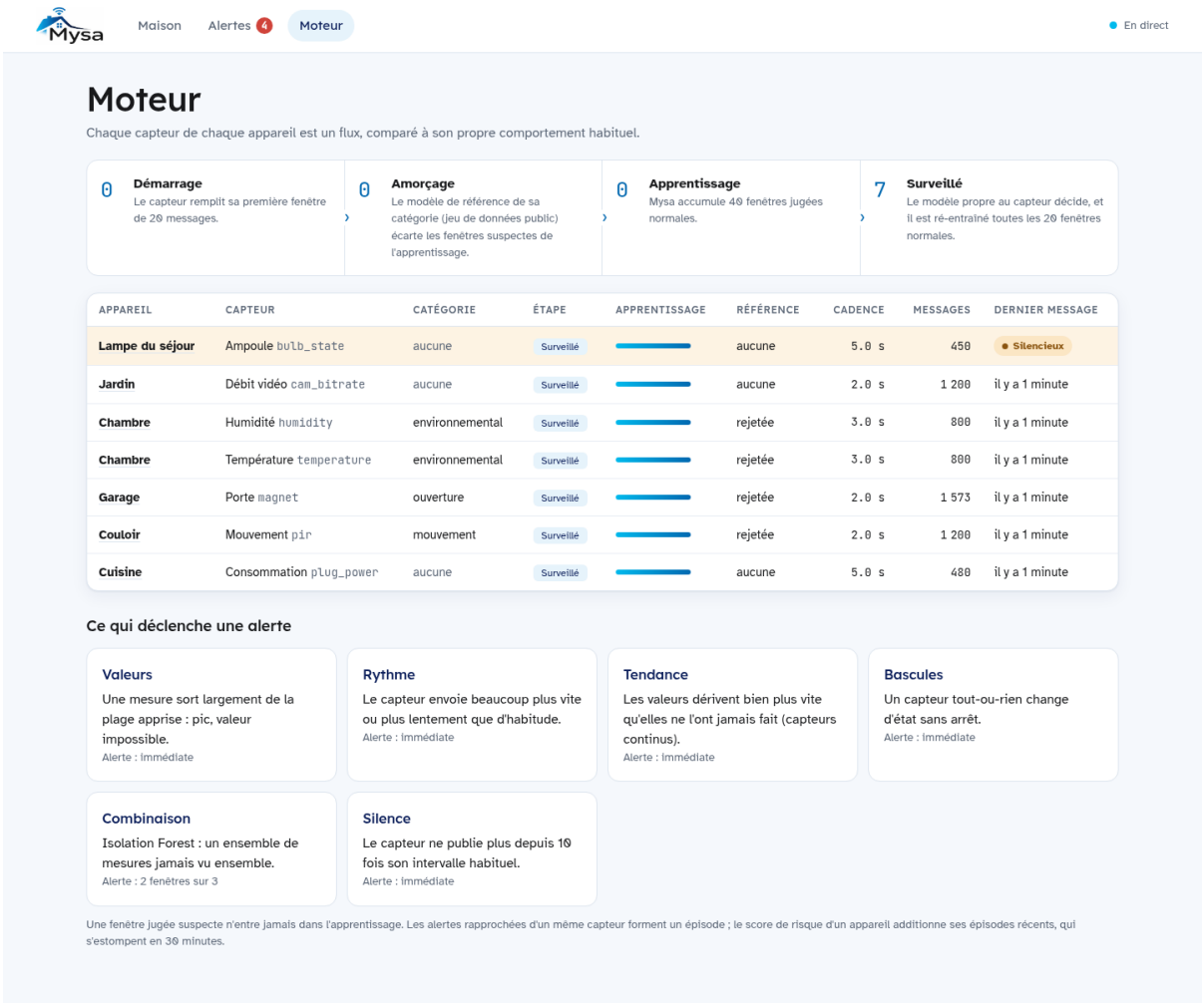


Figure 18 : Dashboard — page « Moteur » (état d'apprentissage de chaque flux)

8.9. Caméra du téléphone

Le téléphone remplace avantageusement une caméra ESP32-CAM : meilleure image, aucun achat. Aucune application n'est installée : le téléphone ouvre dans Chrome la page `https://<hub>:8000/camera/`, qui utilise la caméra arrière comme une application web. HTTPS est obligatoire, Chrome n'autorisant l'accès à la caméra que sur une origine sécurisée ; la CA de Mysa est donc installée sur le téléphone (Paramètres → Sécurité → Installer un certificat CA).

- La page envoie au hub, par WebSocket (`/ws/camera/publish`), chaque image JPEG et une vignette en niveaux de gris très réduite (32 × 24 pixels).
- Le pont caméra de l'API relaie les images vers le dashboard (`/ws/camera/view`), sans aucun enregistrement.
- Le hub compare chaque vignette à la précédente (`engine/vision.py`, numpy seul) : un pixel est « changé » au-delà d'un écart de luminance de 25, et il y a mouvement si

plus de 4 % des pixels ont changé ; la fin du mouvement est déclarée après 8 s de calme, et les deux premières images sont ignorées.

- Au début et à la fin d'un mouvement, le hub publie `home/cam-01/motion` (1 ou 0) avec le compte dédié `mysa-cam`, en précisant si l'alarme est armée : pour le moteur, la caméra est un nœud comme les autres, utilisé par la règle « mouvement filmé » et par les corrélations C-01 et C-02.

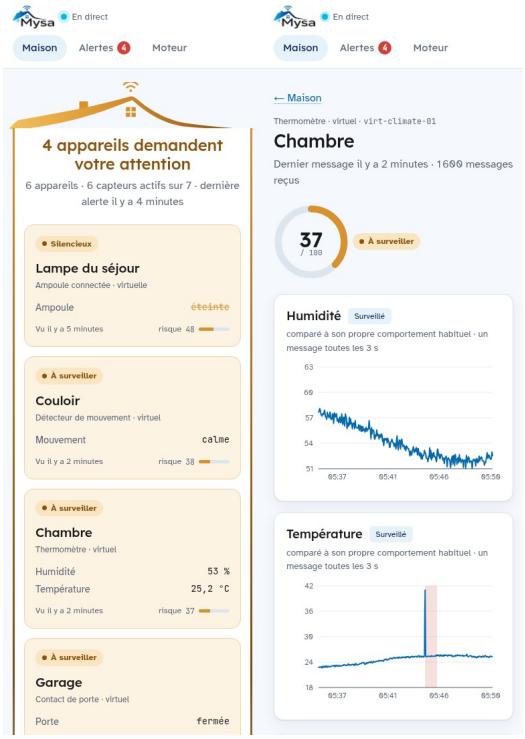


Figure 19 : Page caméra ouverte sur le téléphone

8.10. Notifications

Mysa prévient l'occupant lors des événements marquants : toute alerte de gravité « high » (alarme déclenchée, valeur hors contrat, intrusion détectée par corrélation…) et tout nœud passé hors ligne. Une seule notification est envoyée par épisode, et l'envoi se fait dans un fil d'exécution séparé, sans jamais ralentir l'ingestion. Les trois canaux sont facultatifs : un canal dont les variables ne sont pas renseignées dans `.env` est ignoré.

Tableau 35 : Canaux de notification

Canal	Mise en œuvre	Prérequis
E-mail	SMTP (port 587, STARTTLS ; 465 en SSL)	Mot de passe d'application Gmail (validation en deux étapes)
SMS	Passerelle open source android-sms-gateway en mode local : requête HTTP vers un téléphone Android qui envoie le SMS avec sa carte SIM	Téléphone passerelle à IP fixe, forfait SMS
WhatsApp	Service CallMeBot (requête HTTP)	Clé API obtenue en écrivant au bot ; usage personnel

```
python -c "from dotenv import load_dotenv; load_dotenv(); from engine import
notify; print(notify.dispatch('esp32-sec-01', 'Test de notification Mysa',
'high', 'rule'))"
```

8.11. Déploiement Docker et mode de repli

8.11.1. Hub en trois conteneurs

Le hub complet tourne dans Docker (fichier `docker-compose.yml`) : `broker` (Mosquitto 2, TLS 8883, authentification, ACL), `engine` (moteur d'ingestion) et `api` (API, dashboard et pont caméra en HTTPS 8000). Le moteur et l'API partagent la base et les modèles par des volumes nommés (`mysa-data`, `mysa-models`), qui survivent à un redémarrage.

Sous Windows (Docker Desktop avec WSL2), trois scripts PowerShell suffisent : `deploy\mysa-start.ps1` vérifie que Docker tourne, génère s'ils manquent les certificats et le fichier de mots de passe du broker, puis construit et lance les trois services ; `mysa-stop.ps1` les arrête en conservant les données ; `mysa-reset-demo.ps1` efface la base et les modèles appris en conservant les certificats. Les ports 8000 et 8883 sont ouverts dans le pare-feu Windows (profil privé) et le poste reçoit une adresse IP fixe, inscrite dans le certificat.

```
Copy-Item .env.example .env ; notepad .env      # MYSA_HUB_IP et mots de passe
deploy\mysa-start.ps1
docker compose ps                                # broker, engine et api « running
»
```

[Insérer ici la capture de « docker compose ps » sur le poste Windows (Figure 20)]

Figure 20 : Hub Mysa déployé dans Docker sous Windows

8.11.2. Mode simulation de repli et base de démonstration

Si un composant physique tombe en panne le jour de la démonstration, `scripts/fallback_sim.py` publie à sa place des mesures réalistes, avec les mêmes topics et le même compte (donc conforme à l'ACL), et pose le drapeau retenu `home/<id>/sim = 1` : le dashboard affiche clairement « Simulé ». Sans réseau ni matériel, `scripts/demo_seed.py` construit une base de démonstration réaliste servie par l'API.

8.12. Lab d'attaque

Les scénarios s'exécutent depuis la machine Kali (VLAN 40), uniquement contre le prototype du projet, dans l'environnement isolé. Deux familles sont distinguées : les tests d'intrusion (TI) vérifient que les mesures préventives résistent et que le moteur détecte ce qui passe ; les scénarios comportementaux (SC) vérifient que le moteur détecte une déviation quand un

compte légitime est détourné ou un nœud compromis, et sont rejoués de façon reproductible sur les appareils virtuels (9.3).

Tableau 36 : Tests d'intrusion

Réf.	Scénario	Risque	Outils	Défense attendue de Mysa
TI-01	Écoute du trafic MQTT (connexion anonyme, en clair)	R-01	Client MQTT, Wireshark	TLS et authentification obligatoires : bloqué
TI-02	Force brute des identifiants du broker	R-02	Liste de mots de passe, client MQTT (hydra)	Mots de passe forts : bloqué
TI-03	Usurpation d'un nœud, injection de télémétrie et de commandes	R-03	paho-mqtt	ACL du broker (usurpation et commandes bloquées) et règle « valeur hors contrat » (détection)
TI-04	Saturation (inondation de messages)	R-06	paho-mqtt	Enveloppe de rythme du modèle local (détection une fois le modèle appris)
TI-05	Attaque du canal caméra (injection d'un faux mouvement, écoute du flux)	R-04	Client WebSocket	Segmentation réseau : seuls les objets joignent le hub

Tableau 37 : Scénarios comportementaux

Réf.	Scénario	Ce qu'il éprouve
SC-00	Fonctionnement normal	Fausses alertes
SC-01	Injection de valeurs aberrantes sur un flux existant (compte détourné)	Enveloppe de valeurs, règles de contrat
SC-02	Inondation de messages (rythme anormal)	Enveloppe de rythme
SC-03	Dérive progressive de la valeur	Détecteur de tendance face au ré-entraînement glissant
SC-04	Attaque dès l'installation d'un appareil (pendant l'amorçage)	Hypothèse H1 (6.2.7) : référence contre apprentissage depuis zéro
SC-05	Arrêt d'un appareil (silence)	Chien de garde

8.12.1. Organiser le lab

- `attack-lab/install.sh` : installe sur Kali les outils (nmap, mosquitto-clients, hydra, paho-mqtt) ;
- `attack-lab/lab.env` : cible (`TARGET_HOST=192.168.30.20`), certificat de la CA, identifiants des comptes ciblés ;
- `attack-lab/scenarios/` : `ti01_ecoute.py`, `ti02_bruteforce.py`, `ti03_usurpation.py`, `ti04_saturation.py`, `ti05_camera.py` ;
- `attack-lab/run_report.py` : exécute les cinq scénarios, interroge l'API du hub pour savoir si une alerte a été levée et en combien de temps, et écrit le rapport avant / après en Markdown.

8.12.2. Ouvrir temporairement les flux depuis Kali

Le routeur refuse par défaut tout flux du VLAN 40 (8.2.3). Avant un scénario, la règle correspondante est décommentée dans `gns3/openwrt/firewall`, puis le pare-feu est rechargé (`/etc/init.d/firewall restart`) ; elle est recommentée à la fin. On sait ainsi exactement quel chemin a été ouvert pour chaque mesure.

8.12.3. Exécuter et journaliser

```
bash attack-lab/install.sh
set -a; source attack-lab/lab.env; set +a
python3 attack-lab/run_report.py --out attack-lab/report.md
```

Le rapport classe chaque attaque en « bloquée » (elle n'a pas pu s'exécuter), « détectée » (avec le délai jusqu'à la première alerte) ou « exécutée, non détectée » (à analyser). Le délai de détection se calcule comme l'écart entre le début de l'attaque et la première alerte enregistrée.

Chapitre 9 : Test et validation

Ce chapitre présente le protocole de validation et les résultats obtenus. Deux bancs d'essai sont utilisés : le lab d'attaque, contre le hub durci, et un banc hors ligne qui rejoue les scénarios des appareils virtuels directement dans le moteur, avec une base et des modèles temporaires. Les générateurs des appareils virtuels sont des modèles simples choisis par nous : ces mesures éprouvent le moteur de façon reproductible et ne remplacent pas des relevés sur le prototype physique, dont les résultats restent à compléter (9.5).

9.1. Protocole et mesures

- **Environnement :** Python 3.14, scikit-learn 1.9.1, hub Docker (Mosquitto 2) ; appareils virtuels avec vérité terrain (`scripts/virtual_devices.py`) ; 59 tests automatisés au vert.
- **Taux de détection :** part des anomalies injectées ayant déclenché au moins une alerte sur le flux concerné.
- **Fausses alertes :** épisodes d'alerte levés en dehors de toute anomalie injectée ; rapportés à la durée simulée pour obtenir un nombre de fausses alertes par jour.
- **Délai de détection :** durée entre le début effectif d'une anomalie et la première alerte correspondante.
- **Épisodes d'alerte :** alertes consécutives d'un même flux et d'une même source séparées de moins de 60 s, regroupées en un seul épisode, car les fenêtres glissantes multiplient les alertes d'une même déviation (5.6).
- **Répétitions :** chaque scénario est rejoué avec 5 graines aléatoires (`scripts/replay_eval.py --seeds 1-5`) ; la mesure des fausses alertes utilise 4 graines de 6 heures simulées chacune (`scripts/false_alerts.py`).

9.2. Tests d'intrusion

Le rapport du lab a été généré le 29 octobre 2026 contre le hub Docker (validation locale) ; la colonne « Avant » reprend le résultat attendu sur le système non durci, issu de l'analyse du chapitre 4.

Tableau 38 : Résultats des tests d'intrusion

Réf.	Avant (sans Mysa)	Après (Mysa)	Observations
TI-01	Réussie : trafic en clair, connexion anonyme (V-01, V-02)	Bloquée	Connexion anonyme TLS refusée (code MQTT 135) ; connexion en clair refusée
TI-02	Réussie : identifiants	Bloquée	12 mots de passe essayés contre mysa-engine :

Réf.	Avant (sans Mysa)	Après (Mysa)	Observations
	par défaut (V-02)		aucun accepté
TI-03	Réussie : ni ACL ni validation	Détectée (~0 s) et bloquée	Usurpation de esp32-cmf-01 et commande cmd/lock jetées par l'ACL (aucun message reçu) ; télémétrie forgée pir = 999 détectée par la règle « valeur hors contrat »
TI-04	Non détectée : aucune surveillance du rythme	Exécutée, non détectée à froid	300 messages en rafale (~295 msg/s) sur home/esp32-sec-01/pir, flux sans modèle local appris ; détectée en 3 s une fois le modèle appris (SC-02, tableau 39)
TI-05	Réussie : flux caméra accessible	Protégée par la segmentation	Canal caméra atteint quand le port 8000 est ouvert au VLAN 40 (faux mouvement injecté, flux reçu) ; c'est le pare-feu qui protège ce canal

Ces résultats illustrent la défense en profondeur : chiffrement, authentification, ACL, règles, comportement, segmentation. TI-04 et TI-05 montrent aussi les limites d'une couche prise isolément : la saturation n'est vue par le comportement qu'après l'apprentissage, et le canal caméra, non authentifié, dépend entièrement de la segmentation.

[Insérer ici la capture du rapport généré depuis la machine Kali (Figure 21)]

Figure 21 : Rapport du lab d'attaque exécuté depuis la machine Kali

9.3. Évaluation de la détection comportementale

Le tableau 39 compare, sur les mêmes scénarios et les mêmes graines, la version initiale du moteur (Isolation Forest seul, référence qui alerte, seuil de 20 échantillons) et la version actuelle (quatre détecteurs, confirmation des indices faibles, référence en filtre, règles).

Tableau 39 : Détection et fausses alertes par scénario (6 scénarios × 5 graines)

Scénario	Version initiale	Version actuelle
SC-00 fonctionnement normal	63 épisodes de fausse alerte	0
SC-01 valeurs aberrantes	10/10 détectées en 2 s (médiane), 56 épisodes faux	10/10 en 2 s, 0 épisode faux
SC-02 inondation	5/5 en 62 s, 45 épisodes faux	5/5 en 3 s, 1 épisode faux
SC-03 dérive progressive	5/5 en 38 s, 67 épisodes faux	5/5 en 105 s, 1 épisode faux
SC-04 attaque dès l'installation	5/5 (non significatif)	0/5 : avis de la garde de compatibilité
SC-05 silence	5/5 en 20 s	5/5 en 20 s, 0 épisode faux

Reproduire : python scripts/replay_eval.py --seeds 1-5. Vérité terrain des appareils virtuels.

La version actuelle conserve toutes les détections de SC-01, SC-02, SC-03 et SC-05 et supprime presque toutes les fausses alertes : de 231 épisodes faux à 2 sur l'ensemble des runs. La détection de l'inondation devient vingt fois plus rapide grâce à l'enveloppe de rythme. La dérive

est détectée plus tard (105 s contre 38 s) : le détecteur de tendance exige une pente nettement supérieure à celles de l'entraînement, ce qui évite les fausses alertes de la version initiale, dont la « détection » rapide se confondait avec une alerte permanente.

Le résultat de SC-04 mérite une lecture attentive. Dans la version initiale, la référence « mouvement » signalait le PIR virtuel aussi bien en fonctionnement normal (SC-00) que pendant l'attaque : le 5/5 n'est donc pas significatif. Dans la version actuelle, la garde de compatibilité rejette la référence et l'avis émis énonce les deux explications possibles (décalage du dataset ou attaque présente dès le démarrage).

Tableau 40 : Fausses alertes par jour en fonctionnement normal (4 graines, 1 jour simulé par configuration)

Configuration	**Épisodes de fausse alerte**	**Jours simulés**	**Fausses alertes par jour**
Avant : références qui alertent (hypothèse H1)	7	1,0	7,0
Après : références en filtre seul (choix de Mysa)	1	1,0	1,0

Reproduire : python scripts/false_alerts.py ; données : data/runs/false_alerts.csv.

Utiliser les modèles de référence comme filtre de la baseline, et non comme détecteurs, divise les fausses alertes par sept. L'unique fausse alerte restante provient d'un flux sans modèle de référence (débit de la caméra virtuelle), qui apprend depuis zéro. L'objectif de moins d'une fausse alerte par jour est atteint à la limite ; il reste à confirmer sur le prototype physique.

Bilan des hypothèses. H1 n'est pas confirmée : sur ces scénarios, la référence ne détecte pas mieux une attaque présente dès l'installation, car elle est rejetée par la garde sur les flux virtuels comme en fonctionnement normal. H2 est vérifiée : après la bascule, les deux configurations partagent le même modèle local, et la référence, en filtre, n'a pas dégradé la détection. H3 est réfutée lorsque la référence lève des alertes (7 fausses alertes par jour), et vérifiée lorsqu'elle se limite au rôle de filtre (1 par jour). C'est ce constat qui a fixé le rôle définitif de la référence.

[Insérer ici la capture d'un épisode « valeur hors contrat » ou « mouvement sans accès » affiché sur le dashboard pendant la démonstration (Figure 22)]

Figure 22 : Alerte affichée sur le dashboard lors d'une attaque

9.4. Évaluation hors ligne des modèles de référence

Tableau 41 : Métriques des modèles de référence sur TON_IoT

Catégorie	Dataset	Fenêtres d'entraînement	Test normal / attaque	Fausses alertes	Détection
mouvement	IoT_Motion_Light	15 416	3 854 / 3 258	5,2 %	2,1 %
ouverture	IoT_Garage_Door	18 462	4 616 / 3 881	3,6 %	2,6 %
environnemental	IoT_Weather	22 244	5 562 / 4 614	5,0 %	8,4 %

Le taux de fausses alertes est conforme au réglage du modèle (contamination de 5 % pour les références). Le taux de détection est faible dans les trois cas, et inférieur ou voisin du taux de fausses alertes : les modèles ne distinguent pas les attaques étiquetées du dataset. Pour comprendre pourquoi, on mesure pour chaque caractéristique la capacité à séparer fenêtres normales et fenêtres d'attaque (aire sous la courbe ROC, AUC : 0,5 = aucune séparation, 1 = séparation parfaite).

Tableau 42 : Séparation normal/attaque par caractéristique (AUC)

Caractéristique	Type	mouvement	ouverture	environnemental
mean_abs_delta	portable	0,583	0,502	0,538
std_value	portable	0,509	0,501	0,651
unique_ratio	portable	0,502	0,508	0,522
cv_interval	locale	0,625	0,546	0,512
mean_value	locale	0,505	0,501	0,516
mean_interval_s	locale	0,915	0,894	0,900
std_interval_s	locale	0,924	0,720	0,787
Modèle de référence (score)		0,462	0,488	0,689

Tableau 43 : Cadence des lignes, fenêtres normales et d'attaque

Mesure	mouvement	ouverture	environnemental
Lignes à horodatage identique, normal	62,1 %	78,0 %	76,3 %
Lignes à horodatage identique, attaque	17,8 %	22,9 %	38,3 %
Intervalle moyen par fenêtre (médiane), normal	0,68 s	0,16 s	0,53 s
Intervalle moyen par fenêtre (médiane), attaque	2,37 s	1,84 s	1,79 s

Trois constats ressortent. Premièrement, les caractéristiques portables, qui décrivent la dynamique des valeurs, ne séparent pratiquement pas les classes (AUC de 0,50 à 0,58), sauf l'écart-type de la température (0,65) : les attaques de ces datasets n'altèrent pas les valeurs mesurées, ce qui explique la détection quasi nulle. Deuxièmement, la cadence sépare nettement

les classes dans les trois fichiers (AUC de 0,72 à 0,92 ; 0,89 à 0,92 pour l'intervalle moyen). Troisièmement, ce contraste est le même pour trois capteurs physiquement différents, ce qui suggère une propriété de la façon dont les phases normales et d'attaque ont été enregistrées plutôt qu'un effet des attaques sur les capteurs ; c'est une interprétation, non une démonstration. La cadence est de toute façon une caractéristique propre à chaque appareil : elle est exclue des modèles de référence (6.2.5) et relève du modèle local. Ces caractéristiques n'ont pas été sélectionnées en fonction de ces résultats : le jeu portable a été fixé avant leur mesure, et le diagnostic sert à comprendre les données, non à régler le modèle sur les étiquettes d'attaque.

Pour compléter ce diagnostic, des anomalies ont été injectées dans les fenêtres normales de test (tableau 44).

Tableau 44 : Détection d'anomalies injectées par les modèles de référence

Anomalie injectée	**mouvement**	**ouverture**	**environnemental**
Pic de 8 écarts-types	67,9 %	100 %	36,1 %
Pic de 4 écarts-types	14,1 %	21,2 %	8,3 %
Valeur bloquée	100 %	0 %	100 %
Oscillation extrême	100 %	100 %	100 %
Décalage de niveau (+4 écarts-types)	5,2 %	3,6 %	5,0 %
Fausses alertes (rappel)	5,2 %	3,6 %	5,0 %

Reproduire : python scripts/eval_injection.py ; résultats : engine/models/reference/.injection.json.*

Les références voient bien les valeurs figées et les oscillations, partiellement les pics, mais restent aveugles au décalage de niveau, détecté au taux des fausses alertes : une température passant de 23 °C à 40 °C ne change aucune des trois caractéristiques portables. Ce décalage est couvert par l'enveloppe de valeurs du modèle local et par les règles (surchauffe, variation brutale, valeur hors contrat).

9.5. Discussion et limites

Les résultats obtenus valident l'architecture en couches : le durcissement bloque les attaques les plus probables (TI-01, TI-02, usurpation de TI-03), les règles détectent instantanément ce qui est physiquement ou contractuellement impossible (télémétrie forgée de TI-03), et le modèle comportemental détecte, une fois appris, les valeurs aberrantes, l'inondation et la dérive avec très peu de fausses alertes. L'enseignement principal du projet est que la qualité d'un détecteur d'anomalies domestique se joue autant sur les fausses alertes que sur la détection : sur le banc d'essai, la première version détectait presque tout mais aurait été inutilisable, avec des dizaines d'épisodes faux par scénario.

Plusieurs limites doivent être soulignées :

- **Données simulées.** Les résultats de 9.3 portent sur des générateurs simples choisis par nous. Les mesures sur les nœuds physiques (apprentissage d'une nuit, puis scénarios de la démonstration) restent à reporter ; elles pourraient révéler des comportements plus irréguliers que ceux des générateurs.
- **Fenêtre d'exposition.** Avant la bascule (au moins 215 messages par flux, soit une dizaine de minutes pour le PIR et une vingtaine pour le DHT22), seules les règles et la corrélation détectent : TI-04 n'est pas vu à froid.
- **Apport limité des références publiques.** Les références TON_IoT ne servent que de filtre ; l'hypothèse H1 n'est pas confirmée. Une référence enregistrée sur la maquette elle-même et livrée avec le produit serait plus pertinente.
- **Canal caméra.** Le canal WebSocket de la caméra n'est pas authentifié ; sa protection repose sur la segmentation (TI-05) et sur HTTPS.
- **Validation réseau.** La segmentation GNS3 est configurée mais sa validation complète sur le poste Windows (tests d'isolation, scénarios depuis le VLAN 40) reste à documenter.
- **Sécurité du firmware.** La signature du firmware et le chiffrement de la mémoire flash de l'ESP32 (V-03, V-05) n'ont pas été activés.
- **Paramètres.** Les seuils (enveloppes, score de risque, corrélation) ont été réglés sur le banc d'essai ; ils devront être recalibrés dans un foyer réel.

[Compléter ici avec les mesures sur le prototype physique (apprentissage de nuit, scénarios de la démonstration) une fois réalisées]

Conclusion

Ce mémoire a présenté la conception et la réalisation de Mysa, une architecture autonome de cybersécurité comportementale destinée aux écosystèmes de maisons intelligentes. La démarche a combiné une analyse de l'existant justifiant le choix d'une approche comportementale, une architecture en couches inspirée de la recommandation ITU-T Y.2060 intégrant une segmentation réseau par VLAN, puis la réalisation d'un prototype complet : deux nœuds ESP32 dont la logique de sécurité fonctionne même sans hub, un téléphone recyclé en caméra, un hub déployé en une commande avec Docker, un moteur de détection, un dashboard, des notifications gratuites et un lab d'attaque automatisé.

Les résultats montrent que le durcissement (TLS, comptes, ACL) bloque les attaques les plus probables, et qu'un moteur en trois couches — règles explicables, modèle comportemental par flux, corrélation entre capteurs — détecte les valeurs forgées, l'inondation, la dérive et le silence d'un appareil, avec deux épisodes de fausse alerte seulement sur l'ensemble des scénarios rejoués et une fausse alerte par jour en fonctionnement normal. Ils mettent aussi en évidence les limites de l'approche : la fenêtre d'exposition initiale propre à tout apprentissage de baseline, couverte par les règles mais pas par le comportement, et l'apport limité des données publiques, cantonnées au rôle de filtre de la baseline.

Plusieurs perspectives se dégagent de ce travail. Sur le plan technique : remplacer les références TON_IoT par une référence enregistrée sur la maquette, exploiter CICIoT2023 pour étendre la détection aux flux réseau, essayer des détecteurs incrémentaux (River), signer le firmware et permettre sa mise à jour à distance, authentifier le canal caméra, et déployer la segmentation sur un routeur OpenWrt physique. Sur le plan applicatif, une évolution vers un déploiement en conditions réelles, au sein d'un foyer effectif plutôt que d'un prototype contrôlé, permettrait de confronter l'approche comportementale à la variabilité d'un usage domestique non scénarisé ; le fonctionnement entièrement local, les notifications par SMS et le matériel abordable en font une piste crédible pour le marché malgache, sous réserve d'une résistance au délestage (onduleur, reconnexion automatique) et d'une installation sans reflashage.

Références bibliographiques

[1] ANSSI, EBIOS Risk Manager – Le guide, 2018, Agence nationale de la sécurité des systèmes d'information, Paris

[2] ISO/IEC 27005:2022, Information security, cybersecurity and privacy protection — Guidance on managing information security risks, 2022, ISO, Genève

[3] UIT-T, Recommandation Y.2060 – Vue d'ensemble de l'Internet des objets, 2012, Union internationale des télécommunications, Genève

[4] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash, Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications, 2015, IEEE Communications Surveys & Tutorials, vol. 17, n° 4, pp. 2347-2376

[5] oneM2M, TS-0001 – Functional Architecture, oneM2M Partners Type 1

[6] ETSI, TS 102 690 – Machine-to-Machine communications (M2M); Functional architecture, version 2.1.1, 2013, ETSI, Sophia Antipolis

[7] OASIS, MQTT Version 3.1.1, OASIS Standard, 2014

[8] Z. Shelby, K. Hartke, C. Bormann, The Constrained Application Protocol (CoAP), RFC 7252, 2014, IETF

[9] R. Fielding, M. Nottingham, J. Reschke, HTTP Semantics, RFC 9110, 2022, IETF

[10] F. T. Liu, K. M. Ting, Z.-H. Zhou, Isolation Forest, 2008, Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413-422

[11] F. Pedregosa et al., Scikit-learn: Machine Learning in Python, 2011, Journal of Machine Learning Research, vol. 12, pp. 2825-2830

[12] S. J. Pan, Q. Yang, A Survey on Transfer Learning, 2010, IEEE Transactions on Knowledge and Data Engineering, vol. 22, n° 10, pp. 1345-1359

[13] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems, 2020, IEEE Access, vol. 8, pp. 165130-165150

[14] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A. A. Ghorbani, CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment, 2023, Sensors, vol. 23, n° 13, article 5941

Références webographiques

[15] https://firewalla.com, Firewalla, présentation des produits, consulté le 20 octobre 2026

[16] https://www.bitdefender.com, Bitdefender BOX, consulté le 20 octobre 2026

[17] https://www.fing.com, Fingbox, consulté le 20 octobre 2026

[18] https://docs.platformio.org, PlatformIO Documentation, consulté le 20 octobre 2026

[19] https://docs.espressif.com/projects/esp-idf, ESP-IDF Programming Guide, consulté le 20 octobre 2026

[20] https://micropython.org, MicroPython, consulté le 20 octobre 2026

[21] https://mosquitto.org, Eclipse Mosquitto, consulté le 20 octobre 2026

[22] https://riverml.xyz, River : machine learning for streaming data, consulté le 20 octobre 2026

[23] https://fastapi.tiangolo.com, FastAPI, consulté le 20 octobre 2026

[24] https://www.sqlite.org, SQLite Documentation, consulté le 20 octobre 2026

[25] https://react.dev, React, consulté le 20 octobre 2026

[26] https://recharts.org, Recharts, consulté le 20 octobre 2026

[27] https://www.vmware.com, VMware Workstation, consulté le 20 octobre 2026

[28] https://www.gns3.com, GNS3, consulté le 20 octobre 2026

[29] https://openwrt.org/docs, OpenWrt Documentation, consulté le 2 novembre 2026

[30] https://www.kali.org/docs, Kali Linux Documentation, consulté le 20 octobre 2026

[31] https://github.com/Reinitha18/Mysa, Dépôt du projet Mysa, consulté le 2 novembre 2026

[32] https://research.unsw.edu.au/projects/toniot-datasets, TON_IoT datasets, consulté le 20 octobre 2026

[33] https://www.unb.ca/cic/datasets/iotdataset-2023.html, CICIoT2023 dataset, consulté le 20 octobre 2026

[34] https://docs.docker.com, Docker Documentation (Docker Desktop, Compose), consulté le 2 novembre 2026

[35] https://github.com/capcom6/android-sms-gateway, SMS Gateway for Android, consulté le 2 novembre 2026

[36] https://www.callmebot.com/blog/free-api-whatsapp-messages/, CallMeBot – Free WhatsApp API, consulté le 2 novembre 2026

Glossaire

Acquittement : action de l'utilisateur indiquant qu'un épisode d'alerte a été vu et traité.

ACL (liste de contrôle d'accès) : ensemble de règles définissant, pour chaque compte du broker, les topics qu'il peut lire ou écrire.

Amorçage à froid : stratégie consistant à protéger la détection d'un nouvel appareil à l'aide d'un modèle de référence plutôt que de partir d'un modèle vide, afin de réduire la fenêtre d'exposition initiale.

Baseline comportementale : profil d'activité normal établi pour un flux de télémétrie à partir de son observation continue.

Broker MQTT : serveur intermédiaire assurant la réception et la redistribution des messages publiés par les appareils selon le modèle publication/abonnement.

Catégorie d'appareil : regroupement de capteurs de même nature (mouvement, ouverture, environnemental) auquel est associé un modèle de référence.

Chien de garde : mécanisme qui signale un flux périodique devenu silencieux par rapport à son rythme habituel.

Compte détourné : compte légitime d'un appareil utilisé par un attaquant pour publier de fausses mesures.

Conteneur (Docker) : environnement d'exécution isolé et reproductible, regroupant une application et ses dépendances.

Corrélation : détection qui croise plusieurs capteurs pour reconnaître un scénario qu'aucun capteur ne révèle seul.

Dernière volonté (Last Will) : message déclaré par un client MQTT à sa connexion, que le broker publie lui-même si ce client disparaît.

Empoisonnement de la baseline : introduction, pendant l'apprentissage, de mesures anormales qui sont alors apprises comme normales.

Épisode d'alerte : ensemble d'alertes consécutives rapprochées d'un même flux, regroupées en une seule déviation.

Fausse alerte : alerte levée en l'absence de toute attaque ou déviation réelle.

Fenêtre d'exposition : période initiale, propre à tout système d'apprentissage, durant laquelle aucune détection comportementale fiable n'est encore possible faute de baseline établie.

Fenêtre glissante : suite de messages consécutifs (20 dans le prototype) sur laquelle sont calculées les caractéristiques analysées.

Flux : suite des messages d'un capteur donné d'un appareil donné ; unité d'analyse du moteur.

Isolation Forest : algorithme non supervisé de détection d'anomalies qui isole les observations atypiques par partitionnements aléatoires successifs.

Modèle de référence : modèle de détection pré-entraîné sur une catégorie d'appareils à partir de données publiques, utilisé pendant l'amorçage.

Modèle local : modèle entraîné sur la télémétrie réelle d'un flux donné.

Moindre privilège : principe selon lequel chaque compte ne reçoit que les droits strictement nécessaires à sa fonction.

Pare-feu inter-VLAN : ensemble de règles encadrant les flux autorisés entre deux segments réseau distincts.

Règle explicable : détection fondée sur une condition simple et lisible (par exemple une valeur physiquement impossible), sans apprentissage.

Scénario comportemental : test vérifiant que le moteur détecte une déviation, par exemple celle d'un compte détourné ou d'un nœud compromis.

Score de risque : indicateur synthétique, de 0 à 100, traduisant le niveau d'anomalie récemment observé pour un appareil donné.

Segmentation réseau : division d'un réseau en sous-segments isolés (VLAN) afin de restreindre la propagation du trafic et des menaces.

Télémétrie : ensemble des données mesurées et transmises par un appareil connecté pour décrire son activité.

Test d'intrusion : tentative d'attaque contrôlée, menée contre le prototype pour vérifier que les mesures préventives résistent.

TLS (Transport Layer Security) : protocole de chiffrement et d'authentification des échanges entre les appareils et le hub.

Transfer learning : technique d'apprentissage automatique consistant à réutiliser un modèle déjà entraîné sur une tâche proche comme point de départ pour une nouvelle tâche.

VLAN (réseau local virtuel) : segment logique d'un réseau physique permettant d'isoler des groupes d'appareils entre eux.

Annexes

Les annexes regroupent des éléments utiles à la compréhension de la réalisation, disponibles dans le dépôt du projet [31].

- **Annexe A :** configuration du broker et du réseau (`broker/mosquitto.conf`, `broker/acl.conf`, `gns3/openwrt/*`, `gns3/flux_matrix.md`).
- **Annexe B :** extraits du code du moteur de détection (`engine/rules.py`, `engine/correlation.py`, `engine/model.py`, `engine/analyzer.py`).
- **Annexe C :** firmware des nœuds et guide de câblage de la maquette (`firmware/`, `docs/Mysa_guide_cablage_maquette.pdf`).
- **Annexe D :** guides de déploiement (`docs/deploiement_windows.md`, `docs/gns3.md`, `docs/https.md`, `docs/notifications.md`, `docs/kali.md`).
- **Annexe E :** catalogue des scénarios d'attaque et rapport avant/après (`attack-lab/`), scénario de démonstration et liste de vérification du jour J (`docs/script_demo.md`, `docs/checklist_jour_j.md`).

Table des matières

Curriculum Vitae.....
Sommaire.....
Remerciements.....
Liste des figures.....
Liste des tableaux.....
Liste des abréviations.....
Introduction générale.....
Partie I : PRESENTATIONS.....
Chapitre 1 : Présentation de l'ENI.....
1.1. Information d'ordre général.....
1.2. Missions et historique.....
1.3. Organigramme institutionnel.....
1.4. Domaine de spécialisation.....
1.5. Architecture des formations pédagogiques.....
1.6. Relation de l'ENI avec les organismes externes.....
1.7. Débouchés professionnels des diplômés.....
1.8. Ressources humaines.....
Chapitre 2 : Présentation de l'établissement d'accueil.....
2.1. Description et statuts.....
2.2. Activités.....
2.3. Contacts et adresses.....
2.4. Partenaires.....
2.5. Organigramme.....
Chapitre 3 : Description du projet.....
3.1. Formulation.....
3.2. Objectif et besoins de l'utilisateur.....
3.3. Moyens nécessaires à la réalisation du projet (humains, matériels, logiciels).....
3.3.1. Moyens humains.....
3.3.2. Moyens matériels.....
3.3.3. Moyens logiciels.....
3.4. Résultats attendus.....
3.5. Chronogramme de travail.....
PARTIE II : ANALYSE ET CONCEPTION.....
Chapitre 4 : Analyse préalable.....
4.1. Analyse de l'existant.....

4.1.1. Collecte des données existantes.....
4.1.2. Analyse des technologies et processus.....
4.1.3. Recueil des risques et vulnérabilités.....
4.1.4. Cartographie de l'écosystème.....
4.1.5. Sessions de feedback.....
4.1.6. Rapport d'analyse et recommandations.....
4.1.7. Mise à jour continue.....
4.2. Critique de l'existant (points forts et points faibles).....
4.3. Proposition de solutions.....
Chapitre 5 : Modèle et architecture de l'IoT.....
5.1. Cadre conceptuel et fondements du modèle IoT.....
5.2. Conception et rôle de la couche perception (capteurs et dispositifs).....
5.3. Technologies de connectivité et protocoles de transmission dans la couche réseau.....
5.4. Modélisation schématique d'une architecture IoT appliquée.....
5.5. Analyse comparative des modèles de référence normalisés.....
5.6. Limites techniques et défis d'ingénierie de l'architecture IoT.....
Chapitre 6 : Études et choix des outils.....
6.1. Outils utilisés.....
6.1.1. Outils de développement firmware.....
6.1.2. Protocole et broker de communication IoT.....
6.1.3. Outils d'apprentissage automatique pour la détection d'anomalies.....
6.1.4. Framework backend et API.....
6.1.5. Stockage de la télémétrie.....
6.1.6. Framework frontend et visualisation.....
6.1.7. Environnement de virtualisation, de déploiement et de simulation réseau.....
6.1.8. Outils de test d'intrusion.....
6.1.9. Notifications.....
6.1.10. Outils de gestion de projet et de version.....
6.2. Données et stratégie d'amorçage à froid.....
6.2.1. Problème : le démarrage à froid.....
6.2.2. Datasets publics retenus.....
6.2.3. Principe : un modèle par catégorie, jamais de mélange.....
6.2.4. Association capteur–catégorie.....
6.2.5. Caractéristiques portables et caractéristiques locales.....
6.2.6. Garde-fous et limites.....
6.2.7. Protocole d'évaluation.....
6.2.8. Synthèse du choix technologique.....

PARTIE III : RÉALISATION.....
Chapitre 7 : Présentation de l'environnement.....
7.1. Prototype de maison intelligente.....
7.2. Plateforme d'attaque.....
7.3. Présentation de Mysa.....
Chapitre 8 : Installation et configuration.....
8.1. Broker MQTT sécurisé.....
8.1.1. Installer Mosquitto.....
8.1.2. Créer l'autorité de certification privée et le certificat du hub.....
8.1.3. Créer un compte par appareil.....
8.1.4. Appliquer la configuration et les règles d'accès.....
8.1.5. Vérifier la sécurisation.....
8.2. Segmentation réseau en VLAN.....
8.2.1. Construire la topologie sous GNS3.....
8.2.2. Créer les interfaces VLAN sur le routeur.....
8.2.3. Filtrer les flux inter-VLAN.....
8.2.4. Vérifier l'isolation.....
8.3. Firmware des nœuds ESP32.....
8.3.1. Couche réseau commune.....
8.3.2. Nœud sécurité et accès (esp32-sec-01).....
8.3.3. Nœud confort (esp32-cmf-01).....
8.4. Ingestion MQTT et stockage.....
8.4.1. Préparer l'environnement Python.....
8.4.2. Le schéma de stockage.....
8.4.3. Lancer le moteur d'ingestion.....
8.5. Moteur de détection.....
8.5.1. Règles explicables.....
8.5.2. Corrélation entre capteurs.....
8.5.3. Caractéristiques et modèle local à quatre détecteurs.....
8.5.4. Orchestration des régimes.....
8.5.5. Présence et chien de garde.....
8.5.6. Vérification de bout en bout.....
8.5.7. Score de risque par appareil.....
8.6. Modèles de référence et amorçage à froid.....
8.6.1. Télécharger les datasets.....
8.6.2. Entraîner un modèle par catégorie.....
8.6.3. Contrôler le résultat.....

8.6.4. Démarrage d'un nouvel appareil.....
8.7. API de consultation et de commande.....
8.8. Dashboard de supervision.....
8.8.1. Initialiser et compiler le projet.....
8.8.2. Vues réalisées.....
8.8.3. Principes d'interface.....
8.9. Caméra du téléphone.....
8.10. Notifications.....
8.11. Déploiement Docker et mode de repli.....
8.11.1. Hub en trois conteneurs.....
8.11.2. Mode simulation de repli et base de démonstration.....
8.12. Lab d'attaque.....
8.12.1. Organiser le lab.....
8.12.2. Ouvrir temporairement les flux depuis Kali.....
8.12.3. Exécuter et journaliser.....
Chapitre 9 : Test et validation.....
9.1. Protocole et mesures.....
9.2. Tests d'intrusion.....
9.3. Évaluation de la détection comportementale.....
9.4. Évaluation hors ligne des modèles de référence.....
9.5. Discussion et limites.....
Conclusion.....
Références bibliographiques.....
Références webographiques.....
Glossaire.....
Annexes.....
Table des matières.....
Résumé.....
Abstract.....

Résumé

Ce mémoire présente Mysa, une architecture autonome de cybersécurité comportementale destinée aux écosystèmes de maisons intelligentes. Face aux limites de la détection par signatures dans un parc d'objets connectés hétérogène, l'approche proposée modélise le comportement normal de chaque flux de télémétrie MQTT au moyen d'un modèle local combinant un Isolation Forest calibré et des détecteurs d'enveloppe, de rythme et de tendance, complété par des règles explicables et une corrélation entre capteurs. Des modèles de référence entraînés sur TON_IoT protègent la baseline pendant l'amorçage. Un prototype associant deux nœuds ESP32, un téléphone servant de caméra, un hub Docker (broker TLS avec ACL, moteur, API, dashboard, notifications), une segmentation en trois VLAN et un lab d'attaque a été réalisé. Les attaques d'écoute, de force brute et d'usurpation sont bloquées, la télémétrie forgée est détectée immédiatement, et le moteur ne produit plus qu'environ une fausse alerte par jour sur le banc d'essai.

Mots clés : amorçage à froid, cybersécurité comportementale, détection d'anomalies, Isolation Forest, maison intelligente, MQTT, segmentation réseau

Abstract

This thesis presents Mysa, an autonomous behavioral cybersecurity architecture designed for smart home ecosystems. Given the limitations of signature-based detection in a heterogeneous connected-device environment, the proposed approach models the normal behavior of each MQTT telemetry stream with a local model combining a calibrated Isolation Forest with envelope, rate and trend detectors, complemented by explainable rules and cross-sensor correlation. Reference models trained on TON_IoT protect the baseline during the cold-start phase. A prototype combining two ESP32 nodes, a phone used as a camera, a Docker hub (TLS broker with ACLs, engine, API, dashboard, notifications), three-VLAN network segmentation and an attack lab was built. Eavesdropping, brute-force and spoofing attacks are blocked, forged telemetry is detected immediately, and the engine produces about one false alert per day on the test bench.

Keywords: anomaly detection, behavioral cybersecurity, cold start, Isolation Forest, MQTT, network segmentation, smart home